



INSI UNIVERSITY

**INSTITUT NATIONAL SUPERIEUR
D'INFORMATIQUE**

**MEMOIRE DE PROJET EN VUE DE LA VALIDATION DE LA
MASTER 2**

Domaine : Science et technologie

Mention : Informatique

Parcours : MSI

Présenté Par : Monsieur RATOLOJANAHARY Tina Hubert

**MIGRATION D'UNE PLATEFORME E-COMMERCE
MONOLITHIQUE VERS UNE ARCHITECTURE
MICROSERVICES**

Soutenue le septembre devant la Commission d'Examen composée de :

Président du Jury : -

Examineur : -

Encadreur : -

Année universitaire : 2024–2025

REMERCIEMENTS

Avant d’entamer ce mémoire, nous souhaitons exprimer nos sincères remerciements à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail et à l’aboutissement de ce projet.

- Au Seigneur Tout-Puissant, pour la force, la santé et l’inspiration qu’Il nous a accordées durant ce parcours universitaire et pour avoir rendu possible l’achèvement de ce mémoire.
- À Docteur RAMASONDRANO Andriamanjaka, directeur de l’Institut National Supérieur d’Informatique (INSI), en reconnaissance de l’environnement d’apprentissage offert et du soutien institutionnel apporté tout au long de nos études.
- À Docteur ...notre encadreur pédagogique, pour son accompagnement, ses conseils constructifs et son implication précieuse dans la conception et la finalisation de ce mémoire.
- Aux enseignants de la filière genie logiciel (GL), pour les connaissances théoriques et pratiques transmises durant cette deuxième année de licence, qui ont jeté les bases nécessaires à ce projet.
- À nos famille et à nos proches, pour leur soutien moral et leurs encouragements constants, qui ont été une source de motivation essentielle durant tout ce parcours.
- À nos camarades de promotion, pour l’entraide et la solidarité partagées tout au long de nos études.
- Enfin, à toutes les personnes qui, de près ou de loin, ont apporté leur aide dans la réalisation de ce mémoire et la conception du projet Jereo.

RESUME

Ce mémoire présente les travaux réalisés dans le cadre d'un stage de fin d'études au sein d'une entreprise spécialisée dans le commerce de produit paramédicaux, concernant la modernisation d'une plateforme e-commerce basée sur une architecture monolithique traditionnelle. Face aux diverses limitations en termes de scalabilité, maintenabilité et agilité de développement, ce projet a pour objectif la conception et l'implémentation d'une architecture microservices moderne.

La méthodologie de migration progressive selon le **Strangler Fig Pattern** a été choisie, permettant une transition en douceur sans interruption de service. L'architecture cible repose sur une décomposition en cinq services principaux : **User Service** pour l'authentification et la gestion des utilisateurs, **Product Service** pour la gestion des produits et de l'inventaire, **Order Service** pour le processus de commande, **Payment Service** pour les transactions sécurisées, et **Notification Service** pour la gestion des notifications.

Sur le plan technique, la solution implémente une stack moderne incluant **NestJS** pour les microservices backend, avec différentes bases de données adaptées à chaque service (**MongoDB** pour le service produits et **PostgreSQL** pour les autres), L'**API Gateway** centralise l'authentification JWT, le routage et le rate limiting, tandis que **Docker** et **Kubernetes** assurent la containerisation et l'orchestration. La communication inter-services utilise à la fois le synchronisme HTTP/REST et l'asynchronisme via **RabbitMQ**.

Les résultats de la migration démontrent une amélioration significative des performances avec un temps de réponse moyen réduit de 45%, une capacité de montée en charge multipliée par 10, et une réduction de 60% du temps de déploiement. La scalabilité horizontale permet désormais de faire évoluer indépendamment chaque service selon la charge, tandis que la maintenabilité est grandement améliorée grâce au découplage des fonctionnalités.

Ce projet valide l'intérêt des architectures microservices pour les plateformes e-commerce soumises à une croissance rapide et à des exigences élevées de disponibilité. Il fournit également un cadre méthodologique et des bonnes pratiques pour la migration progressive d'applications monolithiques, contribuant ainsi à la littérature technique sur les transformations architecturales.

Mots-clés : Architecture Microservices, Migration, E-commerce, NestJS, Docker, Kubernetes, Scalabilité, Strangler Fig Pattern, API Gateway, MongoDB, PostgreSQL

ABSTRACT

This thesis presents the work carried out as part of a final-year internship within a company specialized in the trade of paramedical products, focusing on the modernization of an e-commerce platform based on a traditional monolithic architecture. Faced with various limitations in terms of scalability, maintainability, and development agility, this project aims to design and implement a modern microservices architecture.

The progressive migration methodology following the **Strangler Fig Pattern** was chosen, allowing a smooth transition without service interruption. The target architecture is decomposed into five main services : **User Service** for authentication and user management, **Product Service** for product and inventory management, **Order Service** for order processing, **Payment Service** for secure transactions, and **Notification Service** for notification management.

From a technical perspective, the solution implements a modern stack including **NestJS** for backend microservices, with different databases adapted to each service (**MongoDB** for the product service and **PostgreSQL** for the others). The **API Gateway** centralizes JWT authentication, routing, and rate limiting, while **Docker** and **Kubernetes** ensure containerization and orchestration. Inter-service communication relies on both synchronous HTTP/REST and asynchronous messaging via **RabbitMQ**.

The migration results show a significant improvement in performance, with an average response time reduced by 45%, a 10x increase in load-handling capacity, and a 60% reduction in deployment time. Horizontal scalability now allows each service to evolve independently according to the load, while maintainability is greatly enhanced thanks to the decoupling of functionalities.

This project validates the relevance of microservices architectures for e-commerce platforms facing rapid growth and high availability requirements. It also provides a methodological framework and best practices for the progressive migration of monolithic applications, thus contributing to the technical literature on architectural transformations.

Keywords : Microservices Architecture, Migration, E-commerce, NestJS, Docker, Kubernetes, Scalability, Strangler Fig Pattern, API Gateway, MongoDB, PostgreSQL

SOMMAIRE

Table des matières

REMERCIEMENTS

RÉSUMÉ

ABSTRACT

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES

GLOSSAIRE

LISTE DES TABLEAUX

LISTE DES FIGURES

INTRODUCTION

CHAPITRE I - CONTEXTE GÉNÉRAL DU PROJET

- I.1. Présentation de l'INSI
- I.2. Contexte professionnel et entreprise d'accueil
- I.3 Problématique et justification du projet
- I.4 Objectifs et périmètre du projet
- I.5 Méthodologie et plan de travail

CHAPITRE II - ANALYSE ET CONCEPTION

- II.1 Analyse de l'existant et État de l'Art
- II.2 Fondements Théoriques et Contraintes Distribuées
- II.3 Découpage Métier : Le Domain-Driven Design (DDD)
- II.4 Modélisation des Microservices et Communications
- II.5 Architecture de Persistance et Base de Données
- II.6 Modélisation de la Sécurité : Zero Trust Architecture
- II.7 Modélisation des Flux et Cas d'Usage

CHAPITRE III - RÉALISATION

- III.1 Organisation du Projet et Arborescence
- III.2 Environnement Technologique et Outils
- III.3 Architecture Interne et Patrons de Conception
 - III.3.3 Tableau Récapitulatif des Microservices
- III.4 Développement de l'API Gateway
- III.5 Implémentation des Microservices Core
- III.6 Sécurisation et Gestion des Accès

III.7 Orchestration et DevOps

III.8 Stratégie de Test et Validation

CHAPITRE IV - RÉSULTATS ET ANALYSE

IV.1 Évaluation des performances et de la scalabilité

IV.2 Analyse de la qualité logicielle et maintenabilité

IV.3 Bilan critique et limites de la solution

IV.4 Validation fonctionnelle via Postman

IV.5 Perspectives d'évolutions et améliorations futures

CONCLUSION GÉNÉRALE

BIBLIOGRAPHIE

WÉBOGRAPHIE

ANNEXES

Annexe A — Documentation de l'API REST

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES

ACID	Atomicité, Consistance, Isolation, Durabilité
API	Application Programming Interface
ARSB	Administration Réseaux, Systèmes et Bases de données
AWS	Amazon Web Services
BASE	Basically Available, Soft state, Eventually consistent
BFF	Backend-for-Frontend
CAP	Consistency, Availability, Partition tolerance
CRUD	Create, Read, Update, Delete
DDD	Domain-Driven Design
DTO	Data Transfer Object
DX	Developer Experience
E2E	End-to-End
GL	Génie Logiciel
HTTP	Hypertext Transfer Protocol
I2AD	Intelligence Artificielle, Analyse de Données et Automatisation
IAD	Intelligence Artificielle et Data Science
IC	Infrastructures et Cybersécurités
INSI	Institut National Supérieur d'Informatique
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMD	Licence Master Doctorat
MQ	Message Queue
MSI	Management des Systèmes d'Information
NoSQL	Not only SQL
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
RMQ	RabbitMQ
RPC	Remote Procedure Call
RS256	RSA Signature avec SHA-256
S3	Simple Storage Service (AWS)
SI	Systèmes d'Information
SPOF	Single Point of Failure
SQL	Structured Query Language
TALN	Traitement Automatique du Langage Naturel
TLS	Transport Layer Security

UML	Unified Modeling Language
URL	Uniform Resource Locator

GLOSSAIRE

Terme	Définition
Microservices	Architecture logicielle où une application est décomposée en petits services indépendants, chacun responsable d'une fonctionnalité spécifique.
Monolithe	Architecture logicielle où toutes les fonctionnalités d'une application sont regroupées dans une seule et même base de code.
API Gateway	Composant qui agit comme un point d'entrée unique pour les clients, gérant les requêtes vers les différents microservices.
Containerisation	Technique de virtualisation légère qui permet d'exécuter des applications dans des environnements isolés appelés conteneurs.
Orchestration	Gestion automatisée du déploiement, de la mise à l'échelle et de la gestion des conteneurs.
Scalabilité	Capacité d'un système à gérer une augmentation de la charge en ajoutant des ressources.
Strangler Fig Pattern	Méthode de migration progressive d'une application monolithique vers une architecture microservices en remplaçant progressivement les parties du monolithe par des microservices.

LISTE DES TABLEAUX

2	Les parcours en Licence et Master	6
3	Comparaison des systèmes de persistance retenus	19
4	Flux d'événements dans la Saga de commande	21
5	Récapitulatif des microservices de la plateforme Techcare	30
6	Référence synthétique des endpoints de l'API Gateway Techcare	33
7	Endpoints du module Authentification	xiii
8	Endpoints du module Produits	xiii
9	Endpoints du module Catégories	xiv
10	Endpoints du module Inventaire	xiv
11	Endpoints du module Panier	xiv
12	Endpoints du module Commandes	xv
13	Endpoints du module Paiement	xv
14	Endpoints du module Fichiers	xvi
15	Codes d'erreur HTTP retournés par l'API Gateway	xvi

LISTE DES FIGURES

1	Organigramme de l'INSI	3
2	Diagramme de cas d'utilisation du système Techcare	13
3	Diagramme de classe de l'architecture microservices	18
4	Diagramme de séquence de l'authentification utilisateur	22
5	Diagramme d'activité du processus de commande	24
6	Diagramme de séquence du pattern Saga avec compensation	25
7	Structure globale (Monorepo)	27
8	Structure d'un service	27
9	Capture d'écran Postman : Test de réussite de l'authentification	46
10	Capture d'écran Postman : Test de refus d'accès (401 Unauthorized)	47
11	Capture d'écran Postman : Récupération de la liste des produits	48
12	Capture d'écran Postman : Création réussie d'un produit (201 Created)	49
13	Interface RabbitMQ : Pic d'activité sur la file product_manager_queue	50
14	Capture d'écran Postman : Rejet d'une requête mal formée (400 Bad Request)	51
15	Interface S3 : Validation de l'upload de l'image du produit	52

INTRODUCTION

Depuis la création de l'internet, le commerce en ligne a connu une forte croissance et participe désormais de manière significative à l'économie mondiale. Les plateformes e-commerce traditionnelles reposent souvent sur des architectures monolithiques, où toutes les fonctionnalités sont regroupées dans une seule application.

Bien que cette approche ait permis un développement rapide, elle présente plusieurs limitations, notamment en termes de scalabilité, de maintenabilité et d'agilité de développement. C'est pourquoi de nombreuses entreprises se tournent vers des architectures basées sur des microservices, qui offrent une plus grande flexibilité et une meilleure capacité à s'adapter aux évolutions du marché. Cette transition vers les microservices nécessite une réflexion approfondie sur l'architecture logicielle, les choix technologiques et les processus de développement.

De cette analyse, nous avons dégagé une problématique centrale : comment concevoir une plateforme e-commerce basée sur des microservices tout en garantissant une expérience utilisateur fluide et cohérente ?

Pour répondre à cette problématique, l'objectif de ce mémoire est de migrer une architecture monolithique existante vers une architecture basée sur des microservices en utilisant des méthodologies et de techniques modernes de développement logiciel. Nous allons explorer les différentes étapes de cette migration, en mettant l'accent sur les aspects techniques, organisationnels et stratégiques. Nous aborderons également les défis rencontrés lors de cette transition et les solutions mises en place pour les surmonter.

Ce mémoire est structuré en plusieurs chapitres. Le premier chapitre présente le contexte tel que la présentation des parties prenantes et le cadre du projet. Le second chapitre analyse l'architecture existante et la conception de l'architecture cible. Le troisième chapitre détaille le processus de migration, les outils utilisés et les bonnes pratiques adoptées. Enfin, le dernier chapitre évalue les résultats obtenus et propose des pistes d'amélioration pour l'avenir.

CHAPITRE I

CONTEXTE GÉNÉRAL DU PROJET

I.1. Présentation de l'INSI

I.1.1. Information d'ordre général

INSI (Institut National Supérieur d'Informatique) est un institut universitaire situé à Antananarivo, spécialisé dans le domaine de l'informatique. Il accueille principalement des bacheliers ayant choisi cette filière, mais reste également ouvert à toute personne manifestant un intérêt particulier pour les métiers du numérique, à travers son centre de formation professionnelle. L'établissement a été fondé le 17 août 2021 et son siège se trouve à Ankazotokana Ambanidia, Lot VF 32 Ter.

Pour tout renseignement, vous pouvez nous contacter par email à contact@insi.mg ou visiter notre site web : www.insi.mg

INSI University bénéficie du soutien d'entreprises partenaires, dont **Ilo Madagascar SARLU** et **Data Pulse Center**, qui sont également à l'origine de sa création.

I.1.2. Missions et historiques

INSI University a été fondée le 17 août 2021 à Antananarivo, dans un contexte marqué par une forte demande de compétences dans le domaine de l'informatique et des technologies numériques.

Créé à l'initiative de **Ilo Madagascar SARLU** et **Data Pulse Center**, l'Institut a vu le jour avec pour ambition de former une nouvelle génération de professionnels qualifiés, capables de répondre aux besoins du marché local et international.

Depuis sa création, INSI ne cesse d'évoluer en adaptant ses formations aux enjeux technologiques actuels, tout en favorisant un apprentissage pratique, orienté vers les réalités du monde professionnel.

La mission principale de l'INSI est de **former des experts compétents, éthiques et innovants dans le domaine de l'informatique**, capables de contribuer activement au développement économique et numérique de Madagascar.

L'Institut s'engage à :

- Offrir des formations de qualité, à jour des évolutions technologiques,
- Favoriser l'insertion professionnelle de ses apprenants,
- Créer un lien fort entre la formation académique et les besoins réels des entreprises,
- Encourager l'entrepreneuriat et l'innovation technologique,
- Contribuer à la démocratisation de l'accès au savoir numérique.

I.1.3. Organigramme de l'INSI

L'organigramme ci-dessous présente la structure organisationnelle de l'INSI, mettant en évidence les différentes fonctions et responsabilités au sein de l'institut, ainsi que les liens hiérarchiques entre les divers services.

FIGURE 1 – Organigramme de l'INSI
(Source : INSI, 2025)

L'organigramme de l'INSI, en partenariat avec le Collège de Paris, est structuré autour d'une direction générale incarnée par le **Président Directeur Général**. Ce dernier assure la coordination de trois pôles principaux : la **Direction Administrative et Financière**, la **Direction des Études**, et le service des **Relations Internationales et Mobilités**.

Le **Président Directeur Général** assure la direction stratégique de l'institut. Il supervise l'ensemble des activités pédagogiques, administratives et de développement.

Il veille à la qualité de l'enseignement, à la bonne gestion des ressources humaines et matérielles, et représente l'établissement auprès des partenaires et autorités.

La **Direction Administrative et Financière** est chargée de la gestion des affaires internes, notamment à travers deux services essentiels : le **service des scolarités**, qui s'occupe des dossiers étudiants, inscriptions et suivi académique, et les **services supports et stratégiques**, qui assurent un appui logistique, administratif et technologique à l'ensemble de l'institut.

Le pôle des **Relations Internationales et Mobilités** gère les échanges académiques avec l'étranger ainsi que les projets de mobilité étudiante ou enseignante.

La **Direction des Études** supervise quant à elle l'ensemble des programmes pédagogiques et le corps enseignant. Elle travaille en étroite collaboration avec un **Responsable Parcours**, qui coordonne les différentes spécialisations proposées. Cette direction intègre également une cellule dédiée à la **formation professionnelle et à la certification**, en charge des dispositifs de validation des compétences, des certifications professionnelles et des parcours de formation continue. Il comprend également le **Département alternance et emplois**, responsable du suivi des étudiants en contrat d'alternance, des stages professionnels, et de l'insertion professionnelle.

Sous la partie **Responsable Parcours**, plusieurs responsables pédagogiques gèrent les différentes filières de formation : le **parcours tronc commun**, qui regroupe les matières fondamentales partagées par tous les étudiants en début de ciclo ; le **parcours ARSB & IC** ; le **parcours GL & MSI** ; et enfin le **parcours IAD & IAD**.

D'autres structures, bien que non représentés sur cet organigramme, contribuent également de manière significative à la mission de l'INSI.

Le **Conseil de l'École**, il regroupe des membres de l'équipe pédagogique et administrative. Il joue un

rôle consultatif ou décisionnel sur les questions académiques : validation des programmes, suivi des résultats, discipline, méthodologie, etc.

Le **responsable de la formation en ligne**, il conçoit et supervise les dispositifs d'apprentissage à distance. Il s'assure que les contenus en ligne sont accessibles, interactifs et conformes aux standards pédagogiques, tout en accompagnant enseignants et étudiants dans leur utilisation.

L'**Association et Clubs étudiants**, encadré et soutient les initiatives étudiantes, les clubs thématiques (informatique, innovation, sport, culture, etc.), et les événements organisés dans le cadre de la vie étudiante. Cela favorise l'engagement, le développement personnel et les compétences transversales des étudiants.

Cette structure témoigne d'une organisation à la fois pédagogique, administrative et tournée vers l'international et l'employabilité, assurant une formation complète et professionnalisante aux étudiants.

I.1.4. Domaine de spécialisation

INSI University propose des **parcours académiques professionnalisants** dans le domaine de l'informatique et des technologies numériques, articulés autour de licences professionnelles, de masters spécialisés, formations professionnelles continues et d'opportunités de mobilité internationale.

a) Licences professionnelles

Les programmes de licence offrent une formation de base solide, axée sur la pratique, et préparant à une insertion rapide dans le monde professionnel :

- **GL (Génie Logiciel)** : Conception, développement et maintenance d'applications logicielles.
- **ARSB (Administration Réseaux et Systèmes et Base de données)** : Gestion d'infrastructures informatiques, réseaux, serveurs et systèmes.
- **IAD (Intelligence Artificielle et Data Science)** : Introduction aux algorithmes d'IA, au traitement de données et à l'apprentissage automatique.

b) Masters professionnels

Les masters permettent d'approfondir les compétences techniques et managériales, en lien étroit avec les évolutions du secteur :

- **MSI (Management des Systèmes d'Information)** : Pilotage de projets numériques, gouvernance des SI.
- **IC (Infrastructures et Cybersécurité)** : Sécurisation des systèmes, audit, réseaux avancés.
- **I2AD (Intelligence Artificielle, Analyse de Données et Automatisation)** : Applications avancées de l'IA, automatisation des processus et science des données.

c) Formations professionnelles

INSI University propose également des formations professionnelles continues, spécifiquement conçues pour les professionnels déjà en poste souhaitant approfondir leurs compétences ou se spécialiser dans

l'un des domaines proposés. Ces formations sont adaptées aux contraintes des professionnels (horaires flexibles, modules pratiques) et visent à renforcer leur employabilité ou à accompagner leur évolution de carrière.

d) Mobilité internationale & double diplôme

Dans une optique d'ouverture et de reconnaissance internationale, INSI propose à ses étudiants des opportunités de mobilité académique et de double diplôme, à partir de la deuxième année (L2).

- **Format hybride** : 6 mois de formation à Madagascar + 6 mois à l'étranger selon le parcours choisi.
- **Double diplôme en L3 ou M2** : En partenariat avec des institutions internationales telles que :
 - Collège de Paris (France)
 - Keyce Informatique (France)
 - Eskills (Québec, Canada)

Conditions d'éligibilité

- Niveau DALF C1 en français pour la mobilité vers des pays francophones.
- Niveau B2 ou C1 en anglais pour les destinations anglophones.

I.1.5 Architectures des formations pédagogiques

Au sein de l'INSI, une seule mention est proposée : Informatique, déclinée en trois parcours distincts aux niveaux Licence et Master :

Licence	Master
Génie Logiciel (GL)	Management des Systèmes d'Information (MSI)
Administration Réseaux, Systèmes et Bases de données (ARSB)	Infrastructures et Cybersécurité (IC)
Intelligence Artificielle et Data Science (IAD)	Intelligence Artificielle, Analyse de Données et Automatisation (IADA)

TABLE 2 – Les parcours en Licence et Master
(Source : INSI,2025)

L'architecture des études à trois niveaux conformément au système LMD (Licence Master et Doctorat) permet les comparaisons et les équivalences académiques des diplômes au niveau international.

L = Licence (Bac + 3) = L1, L2, L3 = 6 semestres S1 à S6

M = Master (Bac + 5) = M1, M2 = 4 semestres S7 à S10

Le diplôme de licence est obtenu en 3 années des études après Baccalauréat. Et le diplôme de Master est obtenu en 2 ans après obtenu du diplôme de LICENCE.

Le MASTER PROFESSIONNEL est un diplôme destiné à la recherche emploi au terme des études.

Spécificités pédagogiques

- École d'alternance dès la L2 S4 : alternance cours/entreprise pour une meilleure insertion
- Écoles connectées & e-learning : plateforme numérique de formation, cloud intégré, Google Workspace. Cours accessibles en ligne selon les parcours
- Pédagogie active & par projets : hackathons, soutenances exposées, projets réels en entreprise

I.1.6. Relations de l'INSI avec les entreprises et les organismes

L'Institut National Supérieur en Informatique (INSI) entretient des liens étroits avec le monde professionnel. L'INSI collabore activement avec de nombreux organismes publics, semi-publics et privés, tant au niveau national qu'international. Cette collaboration se traduit concrètement par l'organisation régulière de stages obligatoires pour les étudiants, favorisant leur immersion professionnelle et leur employabilité.

Parmi les entreprises et institutions partenaires accueillant nos étudiants en stage chaque année, on peut citer : Data Pulse Center, RELIA Consulting, Code Talent, Bocasay, Minimad, Smartone AI, NOVITY, Eclasia Madagascar, VISEO Groupe, Océane Trade, ACCES Banque,

I.1.7. Partenariat au niveau international

L'INSI s'est inscrit dans une dynamique d'ouverture à l'international grâce à son partenaire fondateur, **ILO Madagascar**, représentant de l'Organisation Internationale du Travail. Cette orientation se renforce aujourd'hui à travers des **partenariats stratégiques avec des institutions académiques et des entreprises internationales**, permettant à l'Institut de proposer une formation alignée sur les standards mondiaux.

L'INSI bénéficie actuellement de **programmes de co-diplomation et de double diplôme** en collaboration avec plusieurs établissements internationaux de renom, tels que :

- **Collège de Paris International**
- **Keyce Informatique** (France)
- **Eskills Academy** (Québec, Canada)
- **École Supérieure Polytechnique d’Amistranana (ESPA)** – pour une synergie régionale renforcée.

Ces partenariats permettent non seulement d’enrichir l’offre pédagogique, mais aussi d’ouvrir des perspectives de **mobilité académique et professionnelle** aux étudiants, en leur offrant une reconnaissance de leurs compétences à l’échelle internationale.

Par ailleurs, à travers les **stages annuels et projets pédagogiques**, l’INSI collabore également avec des entreprises à dimension régionale ou mondiale, telles que :

- **VISEO Groupe**
- **Edoisia Madagascar** (Groupe mauricien)
- **Smartone A1**
- **NOVITY**, entre autres.

Ces relations actives témoignent de l’ancrage de l’INSI dans un **écosystème global**, plaçant l’Institut comme un acteur majeur de la formation en informatique à Madagascar, tourné vers l’innovation et les exigences du marché international.

I.1.8. Parcours professionnels des diplômés

Les diplômés de la Licence en Informatique de l’INSI disposent d’un socle solide de compétences en programmation, systèmes, réseaux, bases de données et développement logiciel. Cette polyvalence leur permet d’intégrer directement le marché du travail ou de poursuivre en Master. À ce niveau, ils peuvent exercer en tant que :

- **Technicien système et réseau**
- **Développeur web / mobile (front-end ou back-end)**
- **Administrateur système junior**
- **Technicien support IT**
- **Intégrateur / testeur logiciel**
- **Assistant en sécurité informatique**
- **Assistant en projets d’IA ou de data (selon profil)**

Les stages annuels en entreprise leur offrent une première expérience professionnelle valorisante dans des structures variées : entreprises technologiques, banques, administrations, startups, ONG, etc.

La formation de Master proposée à l'INSI, grâce à ses parcours spécialisés, forme des professionnels capables d'intervenir sur des projets complexes, innovants et à dimension stratégique. Les débouchés varient selon la spécialisation choisie :

Parcours Ingénierie logicielle, réseaux et systèmes :

- **Ingénieur en développement logiciel / full-stack**
- **Architecte logiciel**
- **Ingénieur système et réseaux**
- **Administrateur cloud / DevOps**
- **Chef de projet informatique**

Parcours Cybersécurité :

- **Expert en sécurité des systèmes d'information**
- **Analyste SOC / CERT**
- **Auditeur en cybersécurité**
- **Responsable de la sécurité IT (RSSI)**

Parcours Intelligence Artificielle et Data :

- **Data Scientist**
- **Data Analyst**
- **Ingénieur en Intelligence Artificielle**
- **Machine Learning Engineer**
- **Développeur d'applications intelligentes**
- **Consultant en IA**
- **Spécialiste en traitement automatique du langage naturel (TALN)**
- **Spécialiste en vision par ordinateur (Computer Vision)**

Grâce aux partenariats internationaux et aux projets en entreprise, les diplômés du Master peuvent également prétendre à des postes dans des structures multinationales, ou poursuivre une carrière académique et de recherche (doctorat, enseignement supérieur).

I.1.9. Ressources humaines

L'INSI s'appuie sur une équipe pédagogique et administrative compétente, dynamique et engagée dans la réussite de ses étudiants. Ses ressources humaines se répartissent comme suit :

- **Président Directeur de l'INSI : Dr RAMASONDRANO Andriamanjaka**

- Responsable parcours ARSB & IC : Mr RATIANARISON Hery Mampionona
- Responsable parcours tronc commun : Dr RAKOTONDRAMANANA R, Sitraka
- Responsable parcours GL & MSI : Mr ANDRIAMIADANA TANTELY ANTOINE
- Responsable parcours IAD & I2AD : par intérim PDG
- Nombre des enseignants : 40
- Nombre des personnels administratives : 04
- Personnels d'appui : 6

I.2. Contexte professionnel et entreprise d'accueil

I.2.1. Présentation de l'entreprise d'accueil

L'entreprise d'accueil de ce projet est **Techcare**, une startup technologique spécialisée dans la vente de produits pharmaceutiques en ligne à Madagascar.

Fondée en 2024, Techcare vise à révolutionner l'accès aux médicaments et aux produits de santé en offrant une plateforme e-commerce conviviale et fiable.

Techcare collabore avec un réseau de pharmacies partenaires à travers le pays pour garantir la disponibilité des produits et assurer une livraison rapide aux clients.

L'entreprise met l'accent sur la qualité du service client, la sécurité des transactions en ligne, et la conformité aux réglementations locales en matière de santé et de commerce électronique.

Grâce à son approche innovante, Techcare aspire à devenir le leader du marché de la pharmacie en ligne à Madagascar, en facilitant l'accès aux soins de santé pour tous les citoyens, même dans les régions éloignées.

I.2.2. Contexte du projet

Le projet de migration d'une architecture monolithique vers une architecture basée sur des microservices s'inscrit dans le cadre de l'évolution technologique de Techcare.

Avec la croissance rapide du commerce en ligne et l'augmentation du nombre d'utilisateurs, l'architecture monolithique actuelle de la plateforme e-commerce de Techcare présente des limitations en termes de scalabilité, de maintenabilité et de flexibilité.

La décision de migrer vers une architecture basée sur des microservices vise à répondre à ces défis en permettant une meilleure gestion des ressources, une adaptation plus rapide aux besoins changeants du marché, et une amélioration de l'expérience utilisateur.

Ce projet est également motivé par la volonté de Techcare d'adopter des pratiques de développement logiciel modernes, favorisant l'innovation et la compétitivité dans un secteur en constante évolution.

I.3 Problématique et justification du projet

La problématique centrale de ce projet est de déterminer comment migrer efficacement une architecture monolithique vers une architecture basée sur des microservices, tout en garantissant une expérience utilisateur fluide et cohérente. Cette migration soulève plusieurs défis techniques, organisationnels et stratégiques, notamment en ce qui concerne la décomposition de l'application monolithique en services indépendants, la gestion des communications entre ces services, et la mise en place de processus de développement adaptés.

La justification de ce projet réside sur ces avantages clés offerts par les microservices :

- Amélioration de la scalabilité : chaque microservice peut être mis à l'échelle indépendamment en fonction de la demande.
- Meilleure maintenabilité : les équipes peuvent travailler sur des services spécifiques sans affecter l'ensemble de l'application.
- Flexibilité accrue : possibilité d'adopter différentes technologies et langages pour différents services.
- Résilience améliorée : une défaillance dans un microservice n'affecte pas nécessairement l'ensemble du système.

I.4 Objectifs et périmètre du projet

L'objectif principal de ce projet est de migrer l'architecture monolithique existante de la plateforme e-commerce de Techcare vers une architecture basée sur des microservices, en utilisant des méthodologies et des techniques modernes de développement logiciel.

Les objectifs spécifiques incluent :

- Analyser l'architecture monolithique actuelle et identifier les composants clés à migrer.
- Concevoir une architecture cible basée sur des microservices, en définissant les services, leurs interactions et les technologies à utiliser.
- Mettre en œuvre la migration en développant et déployant les microservices.
- Tester et valider la nouvelle architecture pour assurer sa performance, sa scalabilité et sa fiabilité.
- Documenter le processus de migration et les leçons apprises pour faciliter les futures évolutions.

la gestion des utilisateurs, l'inventaire et les notifications.

I.5 Méthodologie et plan de travail

Pour mener à bien ce projet de migration complexe, nous avons adopté une démarche structurée combinant des principes agiles et une planification rigoureuse par phases.

I.5.1 Méthodologie de gestion de projet : Scrum

Le choix s'est porté sur la méthodologie **Scrum**, une approche agile permettant de gérer le développement de manière itérative et incrémentale. Cette méthode est particulièrement adaptée à une architecture microservices où chaque service peut être considéré comme une unité fonctionnelle indépendante.

- **Sprints** : Le travail a été découpé en cycles de deux semaines (Sprints). Chaque sprint se termine par la livraison d'une fonctionnalité ou d'un service testable.
- **Cérémonies** : Des réunions de planification (Sprint Planning) et des revues de sprint ont permis d'ajuster les priorités en fonction des défis techniques rencontrés (notamment sur la gestion des communications asynchrones).
- **Backlog** : Les besoins ont été traduits en "User Stories" regroupées dans un backlog produit, priorisées selon leur importance critique pour le système (commençant par l'authentification et la passerelle API).

I.5.2 Plan de travail et étapes de réalisation

Le projet s'articule autour de cinq phases majeures réparties sur la durée du stage :

1. **Phase d'Analyse (Semaine 1-2)** : Étude approfondie du monolithe existant, identification des domaines fonctionnels (Bounded Contexts) et définition de la stratégie de découpage.
2. **Phase de Conception (Semaine 3-4)** : Modélisation des microservices, choix des protocoles de communication (RabbitMQ pour l'asynchronisme, REST pour le synchrone) et conception des schémas de données distribués.
3. **Phase d'Implémentation (Semaine 5-12)** : Développement itératif des microservices (User, Product, Inventory, Notification, File Manager) et de l'API Gateway.
4. **Phase de Tests et Intégration (Semaine 13-14)** : Tests unitaires, tests d'intégration entre services et validation de la sécurité via JWT.
5. **Phase de Déploiement et Documentation (Semaine 15-16)** : Conteneurisation des services, mise en place de l'infrastructure de production et finalisation du rapport technique.

I.5.3 Outils de collaboration et environnement technique

La traçabilité et la qualité du code ont été assurées par l'utilisation d'outils standards de l'industrie :

- **Git** : Pour la gestion de version et le travail collaboratif.
- **Concurrently** : Pour l'orchestration du démarrage des services en environnement de développement.
- **Docker** : Pour la gestion des dépendances d'infrastructure (PostgreSQL, MongoDB, RabbitMQ).
- **Trello/Jira** : Pour le suivi du tableau Kanban et l'avancement des tâches Scrum.

Conclusion partielle

Ce premier chapitre a permis de poser les bases du projet en présentant l'organisme d'accueil, l'institut de formation et surtout les enjeux majeurs liés à la modernisation de la plateforme Techcare. L'analyse des besoins a mis en lumière les limites critiques de l'architecture monolithique actuelle face aux ambitions de croissance de l'entreprise. Cette compréhension du contexte justifie pleinement le passage vers une architecture distribuée, dont la conception détaillée fera l'objet du chapitre suivant.

CHAPITRE II

ANALYSE ET CONCEPTION : VERS UNE ARCHITECTURE DISTRIBUÉE

Ce chapitre expose la réflexion conceptuelle menée pour répondre aux exigences de scalabilité et de haute disponibilité de Techcare. Nous y abordons les fondements théoriques des systèmes distribués, le découpage métier via le Domain-Driven Design (DDD) et la modélisation des flux de données.

II.1 Analyse de l'existant et État de l'Art

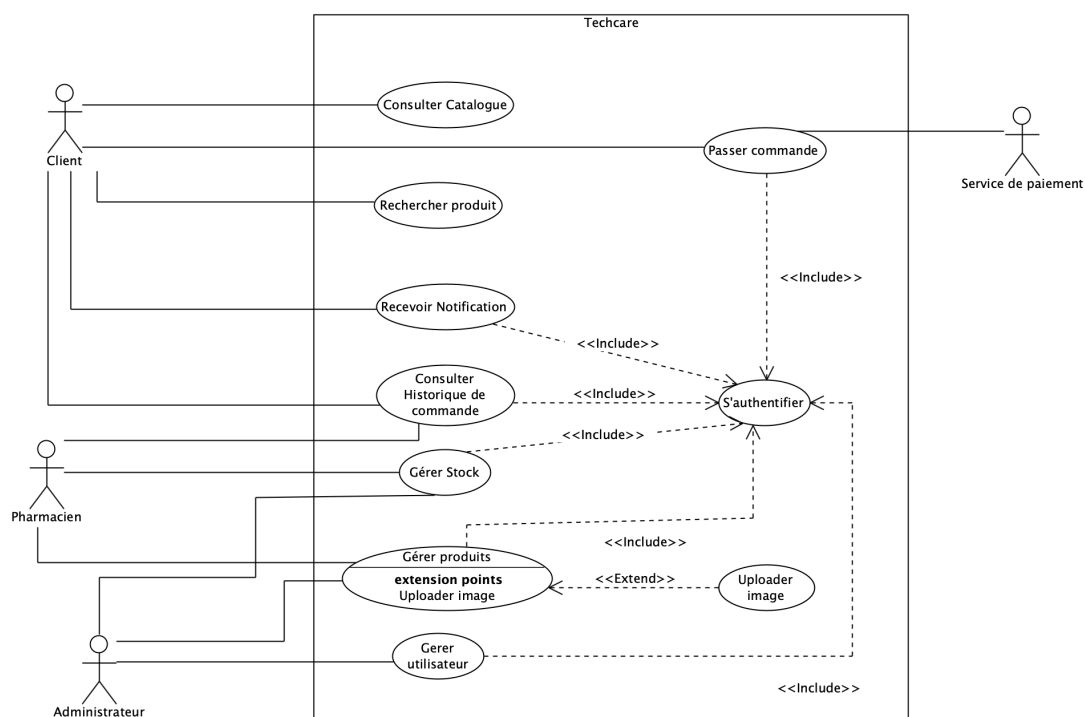


FIGURE 2 – Diagramme de cas d'utilisation du système Techcare

L'analyse de ce diagramme de cas d'utilisation permet de dégager les points suivants :

- **Interactions et Acteurs** : Ce diagramme illustre les interactions fondamentales entre les trois acteurs principaux (Client, Pharmacien et Administrateur) et les fonctionnalités clés, couvrant l'intégralité du cycle de vie médical, de la recherche à la délivrance.
- **Périmètre et Dépendances** : Il met en évidence les relations de dépendance, notamment l'authentification systématique requise pour les actions sensibles, et sert de base pour définir le périmètre fonctionnel du système Techcare.

II.1.1 Évolution des architectures logicielles : Un parcours historique

L'histoire de l'ingénierie logicielle est marquée par une oscillation constante entre centralisation et décentralisation, motivée par la recherche d'un équilibre entre contrôle, performance et agilité.

1. **L'ère des Mainframes (années 1960-1980)** : Le calcul était totalement centralisé sur des machines massives. Les utilisateurs interagissaient via des "terminaux passifs" sans puissance de calcul propre. Bien que robuste, ce modèle offrait une flexibilité nulle et un coût d'entrée prohibitif.
2. **L'ère Client-Serveur (années 1990)** : Avec l'avènement du PC, l'intelligence a été répartie. Le modèle *2-tier* (une application lourde communiquant avec une base de données) a permis l'essor des interfaces graphiques complexes, mais posait des problèmes majeurs de maintenance (chaque mise à jour nécessitait une réinstallation sur chaque poste client).
3. **L'Architecture N-tier et le Monolithe Web (années 2000)** : Pour résoudre les problèmes du client-serveur, le calcul est revenu vers le centre (serveur d'application), mais de manière structurée en couches (Présentation, Métier, Données). C'est l'âge d'or des architectures monolithiques bien structurées, où l'intégralité de la logique réside dans un unique bloc déployable.
4. **SOA (Service-Oriented Architecture)** : Avant les microservices, la SOA a tenté de modulariser l'entreprise. En utilisant des protocoles lourds (SOAP, XML) et un bus de communication central (ESB), elle visait la réutilisabilité. Cependant, la complexité de l'ESB créait souvent un nouveau goulot d'étranglement centralisé.
5. **La révolution des Microservices et du Cloud (2010-Aujourd'hui)** : En réponse aux besoins de "Scale" des géants du web (Netflix, Amazon), le concept de microservices a émergé. Il prône un découplage total, où chaque service est une unité autonome.

Le passage du monolithe aux microservices chez Techcare n'est donc pas une simple tendance, mais l'aboutissement de cette évolution vers la décentralisation contrôlée, répondant à la ****loi de Conway**** : pour que Techcare soit agile, son architecture doit refléter une structure de communication fluide et distribuée.

II.1.2 L'Architecture Monolithique : Analyse du Modèle Traditionnel

L'architecture monolithique constitue le modèle canonique de conception logicielle. Bien qu'aujourd'hui souvent critiquée face à l'essor des microservices, elle a dominé l'industrie pendant des décennies et reste pertinente pour nombre de projets. Pour Techcare, comprendre ses mécanismes est crucial pour justifier la transition.

II.1.2.1 Caractéristiques et Modèle de Déploiement

Dans un système monolithique, l'intégralité de la logique métier, de l'accès aux données et de l'interface est compilée dans un unique artefact exécutable. Cette structure implique plusieurs contraintes

techniques :

- **Couplage Spatial et Temporel** : Tous les composants s'exécutent dans le même espace mémoire et partagent le même cycle de vie. Un redémarrage du serveur pour mettre à jour le module de facturation rend momentanément indisponible le catalogue de médicaments.
- **Gestion des Ressources Partagées** : Le moteur d'exécution (JVM, Node.js, etc.) gère un pool de threads unique. Une boucle infinie ou une fuite de mémoire dans un composant "non critique" peut saturer les ressources et provoquer l'arrêt total du système.
- **Base de données Centralisée** : Le monolithe communique généralement avec un unique schéma relationnel (Shared Database), facilitant les jointures complexes mais créant un point de contention majeur lors de la montée en charge.

II.1.2.2 Atouts Techniques et Opérationnels

Avant d'atteindre ses limites, le monolithe offre des avantages indéniables :

- **Intégrité Transactionnelle (ACID)** : Les opérations inter-modules bénéficient de transactions locales garanties par le moteur de base de données, évitant la complexité des transactions distribuées (Saga).
- **Latence Zéro Inter-Composants** : Les échanges d'informations se font par passage de références en mémoire. Contrairement aux microservices, il n'y a ni sérialisation (JSON/Protobuf), ni latence réseau, ni risque de timeout entre les fonctions.
- **Facilité de Refactoring** : Grâce au typage fort et à la vision globale du code, les outils de l'IDE permettent de renommer ou de restructurer des composants à travers toute l'application en une seule opération sécurisée.

II.1.2.3 Limites Structurelles et le phénomène "Big Ball of Mud"

Pour Techcare, le monolithe a fini par dériver vers ce que Brian Foote et Joseph Yoder appellent une **"Big Ball of Mud"** (grosse boule de boue) : une architecture sans structure apparente où le couplage devient si dense que toute modification devient imprévisible.

- **Lourdeur du Continuous Deployment (CD)** : Le temps de build augmente de manière exponentielle avec la taille du projet. Pour Techcare, déployer un correctif mineur prenait plus de 30 minutes, incluant l'exécution de milliers de tests sans rapport avec le changement.
- **Scalabilité Inefficace** : La scalabilité est obligatoirement "homogène". Pour absorber un pic de trafic sur la recherche de médicaments, nous devons dupliquer l'intégralité du socle (incluant les modules lourds et inutilisés en lecture seule), entraînant un surcoût d'infrastructure inutile.
- **Obsolescence Technologique Subie** : Le système est "prisonnier" de ses choix initiaux. Passer d'une version majeure de framework ou changer de langage est une mission quasi-impossible car elle impacterait des millions de lignes de code interdépendantes.

II.2 Fondements Théoriques et Contraintes Distribuées

II.2.1 Le Théorème CAP : Le dilemme de la distribution

Formulé par Eric Brewer, le théorème CAP stipule que pour un système informatique distribué, il est impossible de garantir simultanément plus de deux des trois propriétés suivantes :

- **Consistance (Consistency)** : Tous les nœuds voient les mêmes données au même moment.
- **Disponibilité (Availability)** : Chaque requête reçoit une réponse (succès ou échec), sans garantie qu'elle contienne la version la plus récente.
- **Partition Tolerance** : Le système continue de fonctionner malgré la perte de messages ou la défaillance d'une partie du réseau.

Pour Techcare, notre choix s'est porté sur une architecture ****AP (Availability and Partition Tolerance)****. Dans un contexte pharmaceutique, il est préférable qu'un client puisse consulter le catalogue de médicaments (quitte à voir un stock légèrement désynchronisé de quelques secondes) plutôt que de se confronter à une erreur 500 bloquante. Nous adoptons donc le modèle de **Consistance Éventuelle (Eventual Consistency)**, où les données se propagent de manière asynchrone via RabbitMQ.

II.2.2 Patterns de Résilience : Prévenir l'effet domino

L'architecture distribuée multiplie les points de défaillance réseau. Pour éviter qu'une panne isolée ne paralyse tout le système, nous avons implémenté :

- **Circuit Breaker (Disjoncteur)** : Ce pattern surveille les échecs d'un service distant. S'ils dépassent un seuil, le disjoncteur "s'ouvre" et redirige immédiatement les appels vers un mode dégradé (ex : cache ou message d'erreur rapide), évitant ainsi d'épuiser les ressources du client en attendant un timeout.
- **Retry Pattern** : Pour les erreurs transitoires (micro-coupures réseau), le système réessaye automatiquement la requête avec une stratégie d'*Exponential Backoff* pour ne pas saturer le service cible.
- **Bulkhead** : Inspiré des compartiments étanches des navires, ce pattern isole les pools de threads par service. Une saturation du *Payment Service* n'impactera pas les performances du *Product Service*.

II.3 Découpage Métier : Le Domain-Driven Design (DDD)

Le Domain-Driven Design est au cœur de notre stratégie de décomposition. Il permet de s'assurer que le code reflète fidèlement la complexité métier.

- **Bounded Contexts (Contextes Délimités)** : C'est la pierre angulaire de notre architecture. Au lieu d'avoir un modèle "Produit" géant, nous avons fragmenté ce concept. Dans le *Catalog*

Context, le produit est défini par son marketing (images, description). Dans l'*Inventory Context*, il est défini par sa logistique (code-barres, emplacement, seuil d'alerte).

- **Ubiquitous Language (Langage Commun)** : Nous avons banni le jargon technique pur pour utiliser les termes des pharmaciens (ex : "Posologie", "Rupture de stock", "Délivrance") directement dans le nommage des classes et des méthodes.
- **Aggregate Roots** : Chaque microservice gère une entité racine (ex : l'Ordre dans le service Commande) qui garantit l'intégrité de ses composants internes.

II.4 Modélisation des Microservices et Communications

II.4.1 API Gateway et Backend-for-Frontend (BFF)

L'API Gateway constitue le point d'entrée unique pour tous les clients du système Techcare. Elle agit comme une couche d'abstraction entre les clients (Web, Mobile) et la complexité interne des microservices.

- **Façade et Routage intelligent** : La Gateway redirige les requêtes entrantes vers les microservices appropriés en fonction de l'URL, masquant ainsi la topologie réelle du réseau interne et les adresses spécifiques de chaque service.
- **Agrégation de requêtes** : Pour optimiser les performances, elle peut fusionner les réponses de plusieurs services en un seul appel (ex : réunir les informations marketing de *ProductService* et les niveaux de stock de *InventoryService*), réduisant ainsi le "bavardage" réseau (*chatter*) et la latence côté client.
- **Gouvernance centralisée** : Elle assure des fonctions transverses (*Cross-Cutting Concerns*) telles que la terminaison SSL, la gestion des CORS, le *Rate Limiting* pour prévenir les abus, et la validation initiale des jetons JWT.
- **Pattern BFF (Backend-for-Frontend)** : Cette approche nous permet de fournir des réponses adaptées aux besoins spécifiques de chaque interface utilisateur. Un client mobile recevra une version allégée des données pour économiser la bande passante, tandis qu'un portail administrateur pourra obtenir des informations plus denses.

II.4.2 Chorégraphie : La force de l'asynchronisme

Pour la communication entre services, nous avons exclu l'orchestration centralisée (qui crée un couplage fort et un SPOF) au profit de la ****Chorégraphie**** via le broker de messages RabbitMQ :

- **Découplage temporel** : Un service peut émettre un événement même si le destinataire est temporairement hors ligne. Le broker garantit la persistance des messages.
- **Scalabilité horizontale** : Plusieurs instances d'un service peuvent consommer la même file d'attente pour traiter les tâches en parallèle.

- **Réduction de la latence** : L'utilisateur reçoit une réponse immédiate ("Commande acceptée"), tandis que le traitement lourd (paiement, notification, mise à jour des stocks) s'effectue en arrière-plan.

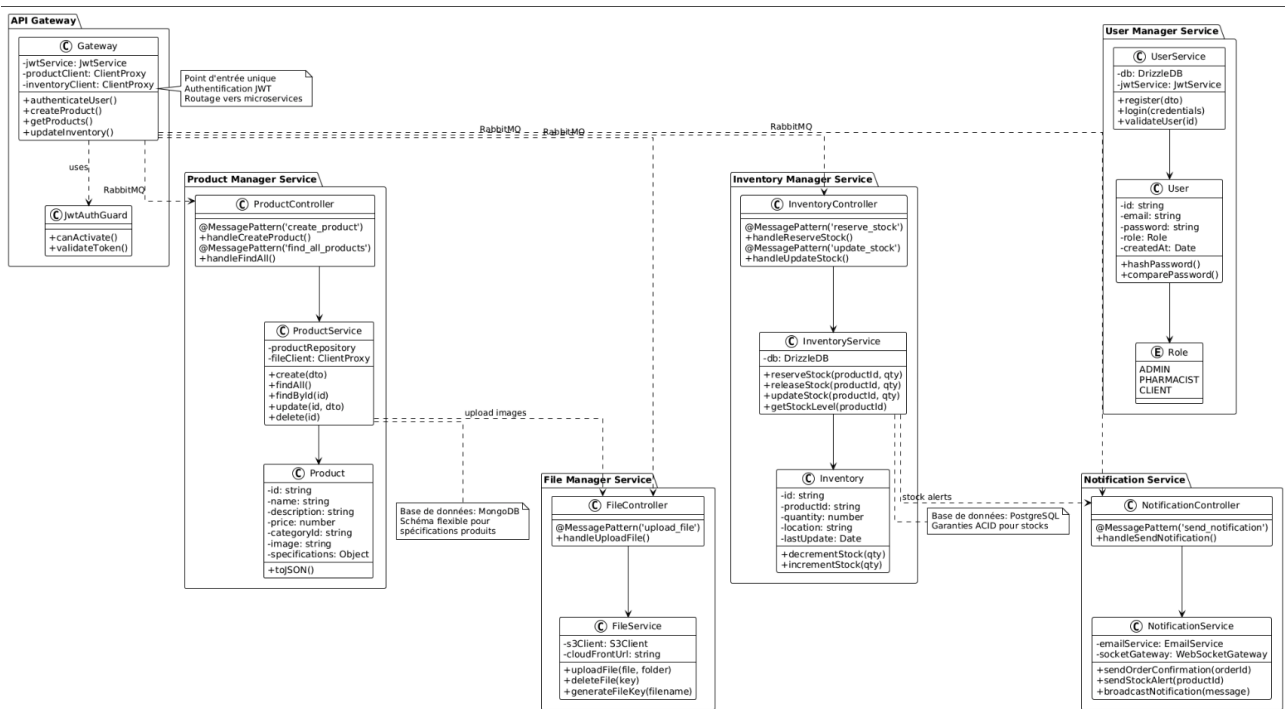


FIGURE 3 – Diagramme de classe de l'architecture microservices

L'analyse de cette architecture de classes révèle deux aspects fondamentaux de la conception :

- **Organisation Domain-Driven (DDD)** : Contrairement au monolithe, les entités sont regroupées par contextes délimités (Catalog, Inventory, Order), garantissant un faible couplage et permettant à chaque domaine de posséder sa propre logique métier indépendante.
- **Flexibilité et Évolutivité** : La structure met en évidence le rôle de l'API Gateway et la séparation des services, ce qui permet au système d'évoluer organiquement et de faciliter l'adoption de nouvelles technologies par service sans impacter l'ensemble du système.

II.5 Architecture de Persistance et Base de Données

Le choix de la **Persistance Polyglotte** répond au besoin de performance :

- **PostgreSQL (ACID)** : Indispensable pour l'Inventory Manager afin d'éviter le "Double Spending" de médicaments.
- **MongoDB (BASE)** : Idéal pour le Product Manager afin de stocker des métadonnées hétérogènes (composition, posologie, photos).

II.5.1 Justification des choix technologiques

Le tableau suivant synthétise les critères de sélection des bases de données :

Critère	PostgreSQL	MongoDB
Modèle de données	Relationnel (schéma fixe)	Documentaire (schéma flexible)
Consistance	ACID stricte	Consistance éventuelle
Cas d'usage Techcare	Stocks, Utilisateurs	Catalogue produits
Performance	Excellent pour transactions	Excellent pour lectures massives

TABLE 3 – Comparaison des systèmes de persistance retenus

II.5.1.1 Justification du choix de PostgreSQL

PostgreSQL a été retenu pour les services nécessitant une **intégrité transactionnelle stricte**. Voici les raisons détaillées de ce choix :

- a) **Garanties ACID indispensables** : Pour le service **Inventory Manager**, la gestion des stocks de médicaments ne tolère aucune incohérence. Les propriétés ACID de PostgreSQL garantissent :
 - **Atomicité** : Une opération de décrémentation de stock est soit totalement effectuée, soit totalement annulée.
 - **Consistance** : Les contraintes de vérification (ex : `quantity >= 0`) sont appliquées.
 - **Isolation** : Les verrous empêchent les réservations doubles simultanées.
 - **Durabilité** : Les modifications sont persistées même en cas de panne.
- b) **Support des contraintes relationnelles** : Le service **User Manager** exploite les relations entre tables. PostgreSQL impose l'intégrité référentielle, évitant les incohérences de données orphelines.
- c) **Performance des jointures complexes** : Les requêtes agrégées (ex : historique de commandes détaillé) bénéficient de l'optimiseur de PostgreSQL, très performant pour les jointures massives.
- d) **Typage strict et sécurité** : L'utilisation de Drizzle ORM avec PostgreSQL offre une sécurité type-safe dès la compilation, réduisant drastiquement les bugs.

II.5.1.2 Justification du choix de MongoDB

MongoDB a été sélectionné pour le service **Product Manager** en raison de la nature hétérogène des produits pharmaceutiques.

- a) **Flexibilité du schéma documentaire** : Les produits médicaux ont des caractéristiques variables (sirops, comprimés, dispositifs médicaux). MongoDB permet de stocker ces variations dans une même collection sans migration de schéma complexe.

- b) **Performance des lectures à grande échelle** : Le catalogue étant majoritairement consulté en lecture, MongoDB excelle grâce à son indexation avancée et son cache mémoire intégré.
- c) **Support natif des données imbriquées** : Un produit contient des tableaux d'images ou de contre-indications. MongoDB stocke ces données nativement, évitant les jointures SQL coûteuses.
- d) **Évolutivité sans migration** : L'ajout de nouveaux types de produits ne nécessite aucune modification de la structure de la base de données existante.

Conclusion sur la persistance (Trade-offs) : Bien que la persistance polyglotte offre une flexibilité maximale, elle impose une complexité opérationnelle accrue. L'équipe de développement doit maîtriser deux langages de requêtage différents et la gestion des sauvegardes (*backups*) devient plus hétérogène, nécessitant des stratégies distinctes pour le relationnel et le documentaire.

II.5.1.3 Justification des technologies applicatives et services tiers

Le choix des outils de développement et des services externes a été guidé par des impératifs d'architecture distribuée et de sécurité de l'information.

- a) **NestJS pour l'architecture modulaire** : Choisi pour son injection de dépendances et son support natif des microservices. Il impose une rigueur indispensable à la maintenabilité du système.
- b) **Stockage Objet (AWS S3) pour la scalabilité** : Centralisation des médias pour garantir des services *stateless*, facilitant ainsi la montée en charge horizontale.
- c) **Délégation de paiement (Stripe)** : Assure la sécurité financière et la conformité PCI-DSS en évitant le stockage de données sensibles en local.

Limites et contraintes identifiées : L'adoption de ces solutions modernes n'est pas sans contreparties. L'utilisation de NestJS impose une courbe d'apprentissage initiale plus raide pour les développeurs habitués à des structures moins rigides. Par ailleurs, la dépendance à des services tiers comme AWS S3 ou Stripe introduit un risque de *Vendor Lock-in* (dépendance vis-à-vis d'un fournisseur) et soumet le système à la disponibilité réseau de ces prestataires externes.

II.5.2 Gestion des transactions distribuées : Pattern Saga

Dans une architecture microservices, l'absence de transaction globale (ACID) inter-services pose un défi majeur. Le pattern **Saga**, introduit par Hector Garcia-Molina et Kenneth Salem en 1987, apporte une solution élégante à ce problème.

Définition formelle

Une Saga est une séquence de transactions locales $T_1, T_2, \dots, T_n$ où chaque transaction T_i possède une transaction compensatoire C_i . Si une transaction T_k échoue, le système exécute les compensations $C_{k-1}, C_{k-2}, \dots, C_1$ dans l'ordre inverse pour annuler les effets des transactions déjà validées.

Les deux approches de Saga

1. Orchestration (Centralisée) Un coordinateur central (Orchestrator) pilote l'exécution des transactions.

- **Avantages** : Visibilité complète du flux, facilité de debugging
- **Inconvénients** : Point de défaillance unique (SPOF)

2. Chorégraphie (Décentralisée) - *Choix retenu pour Techcare* Chaque service écoute des événements et déclenche ses propres actions.

- **Avantages** : Pas de SPOF, meilleure scalabilité
- **Inconvénients** : Complexité accrue du monitoring

Exemple concret : Commande de médicament

Considérons le scénario où un client commande un médicament sur Techcare :

Étapes de la Saga (Happy Path) :

1. **ProductService** : Vérifie la disponibilité du produit → Émet ProductValidated
2. **InventoryService** : Réserve le stock → Émet StockReserved
3. **PaymentService** : Traite le paiement → Émet PaymentCompleted
4. **NotificationService** : Envoie confirmation → Émet OrderConfirmed

Scénario de compensation (Failure Path) : Si le PaymentService échoue à l'étape 3, les compensations suivantes sont déclenchées :

- C_2 : InventoryService libère la réservation de stock
- C_1 : ProductService marque le produit comme disponible
- Notification d'échec envoyée au client

Implémentation via RabbitMQ : Chaque service publie des événements dans des échanges dédiés. Les autres services écoutent les événements qui les concernent via des queues. Cette architecture événementielle garantit la résilience : si un service est temporairement indisponible, les messages restent dans la queue jusqu'à son retour en ligne.

Événement	Émetteur	Écouteur
ProductValidated	ProductService	InventoryService
StockReserved	InventoryService	PaymentService
PaymentFailed	PaymentService	InventoryService
StockReleased	InventoryService	NotificationService

TABLE 4 – Flux d'événements dans la Saga de commande

Ce mécanisme de chorégraphie distribué est illustré techniquement par le diagramme de séquence présenté en section II.7.1 (page 25).

II.6 Modélisation de la Sécurité : Zero Trust Architecture

Dans notre conception, nous n'accordons aucune confiance implicite au réseau interne.

II.6.1 Architecture de sécurité en couches

1. **Authentication Stateless** : Utilisation de JWT signés par algorithme asymétrique (RS256).
2. **Propagation d'identité** : Le jeton est transmis de service en service, permettant un audit complet de chaque action.
3. **Chiffrement End-to-End et en transit** :
 - **Transactions financières** : Cryptage de bout en bout entre le client et Stripe (les données bancaires ne transitent jamais sur nos serveurs).
 - **Données sensibles** : Utilisation de TLS 1.3 pour toutes les communications avec AWS S3 et entre l'API Gateway et les clients.
 - **Sécurisation au repos** : Hachage des mots de passe via l'algorithme Bcrypt dans le User Service.

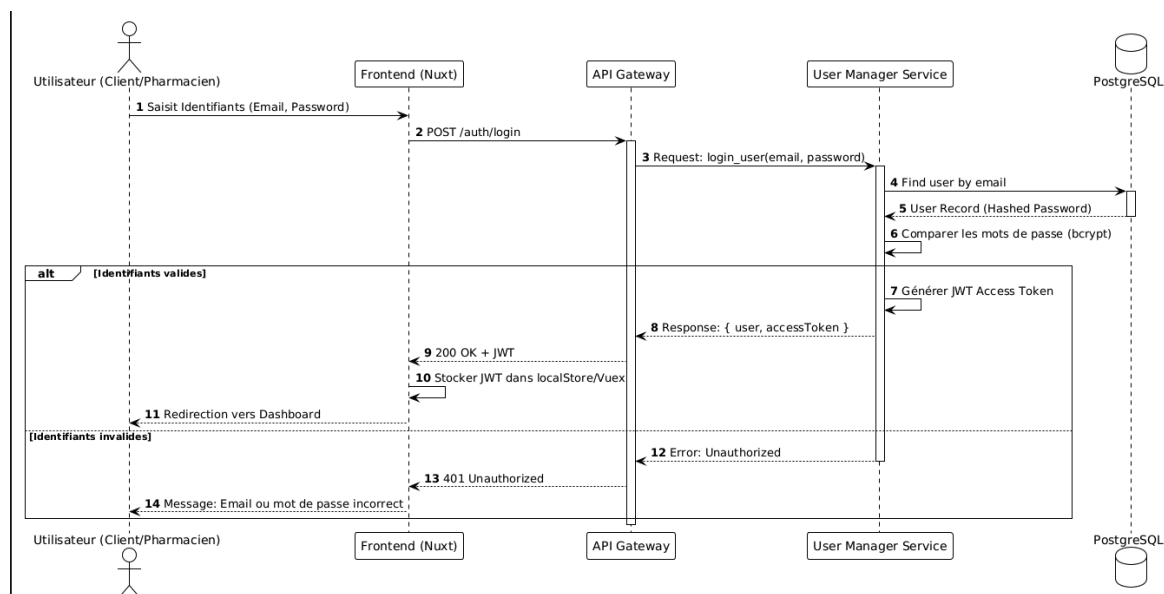


FIGURE 4 – Diagramme de séquence de l'authentification utilisateur

Ce flux d'authentification met en exergue deux piliers de notre stratégie de sécurité :

- **Authentication Stateless avec JWT** : Le diagramme détaille l'utilisation de jetons signés cryptographiquement (RS256) transmis au client, ce qui élimine le besoin de sessions persistantes sur le serveur et facilite la scalabilité horizontale.
- **Validation Décentralisée** : Cette approche délègue la vérification de l'identité à chaque micro-service de manière autonome, supprimant les goulots d'étranglement centraux et renforçant la résilience globale lors de la navigation sur la plateforme.

II.6.2 Gestion des rôles et permissions

Le système implémente un contrôle d'accès basé sur les rôles (RBAC) :

- **Admin** : Accès complet (CRUD produits, gestion stocks, utilisateurs)
- **Pharmacien** : Lecture catalogue, gestion commandes, mise à jour stocks
- **Client** : Consultation catalogue, passage de commandes

II.7 Modélisation des Flux et Cas d'Usage

Pour illustrer l'architecture, prenons le cas d'usage critique : "Passer une commande".

II.7.1 Processus métier de commande

Le flux se déroule ainsi :

1. Le client envoie une requête HTTP POST à l'API Gateway
2. La Gateway vérifie le JWT et route vers le Product Manager via RabbitMQ
3. Le Product Manager valide la disponibilité
4. Un événement "ProductReserved" est publié
5. L'Inventory Manager écoute cet événement et décrémente le stock
6. Une notification est envoyée au client via Socket.io

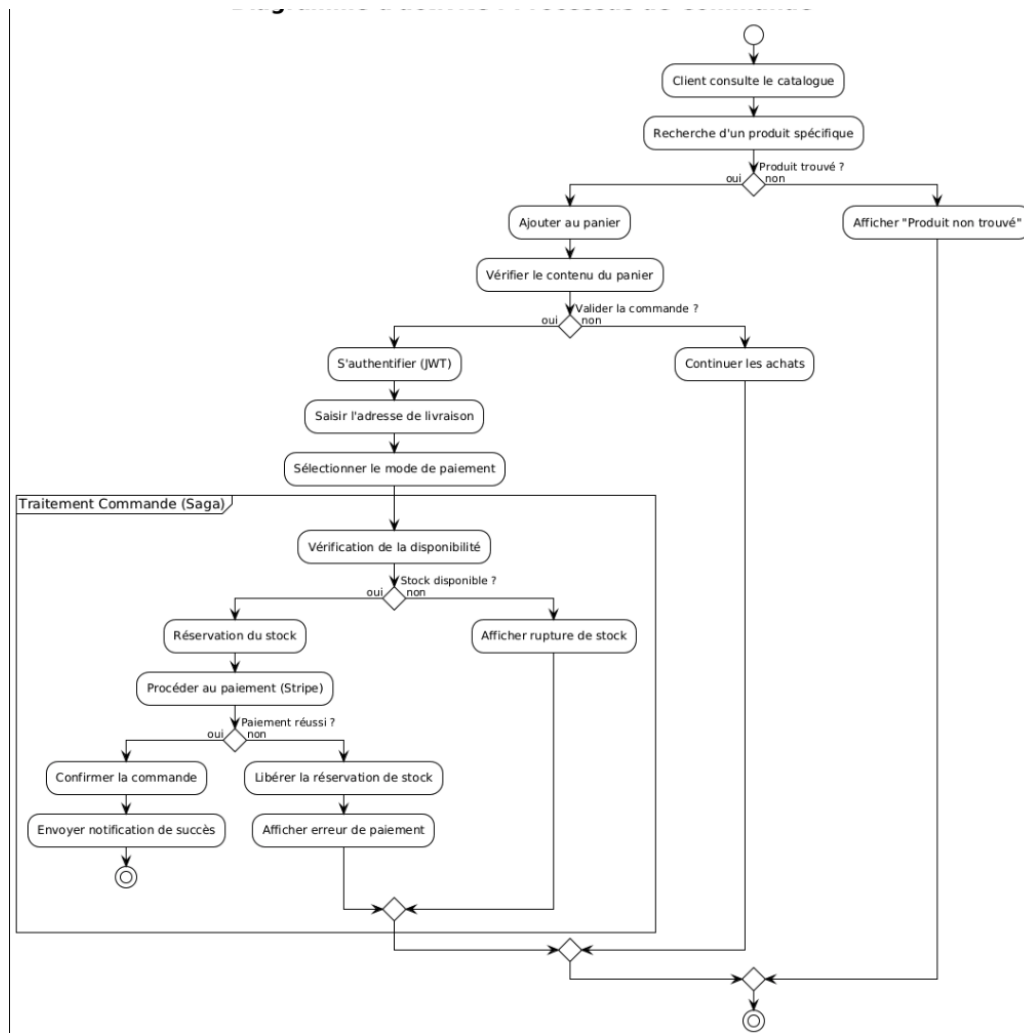


FIGURE 5 – Diagramme d'activité du processus de commande

Ce processus métier peut être analysé selon deux axes principaux :

- **Logique Métier et Décisions** : Ce flux retrace le cheminement critique d'une commande, soulignant les points de décision comme la validation des stocks ou le succès du paiement, et les bifurcations en cas d'erreur.
- **Optimisation de l'Expérience** : La visualisation de ces étapes a permis d'identifier les zones de contact utilisateur, facilitant l'intégration de notifications en temps réel pour informer le client de l'état d'avancement de son traitement.

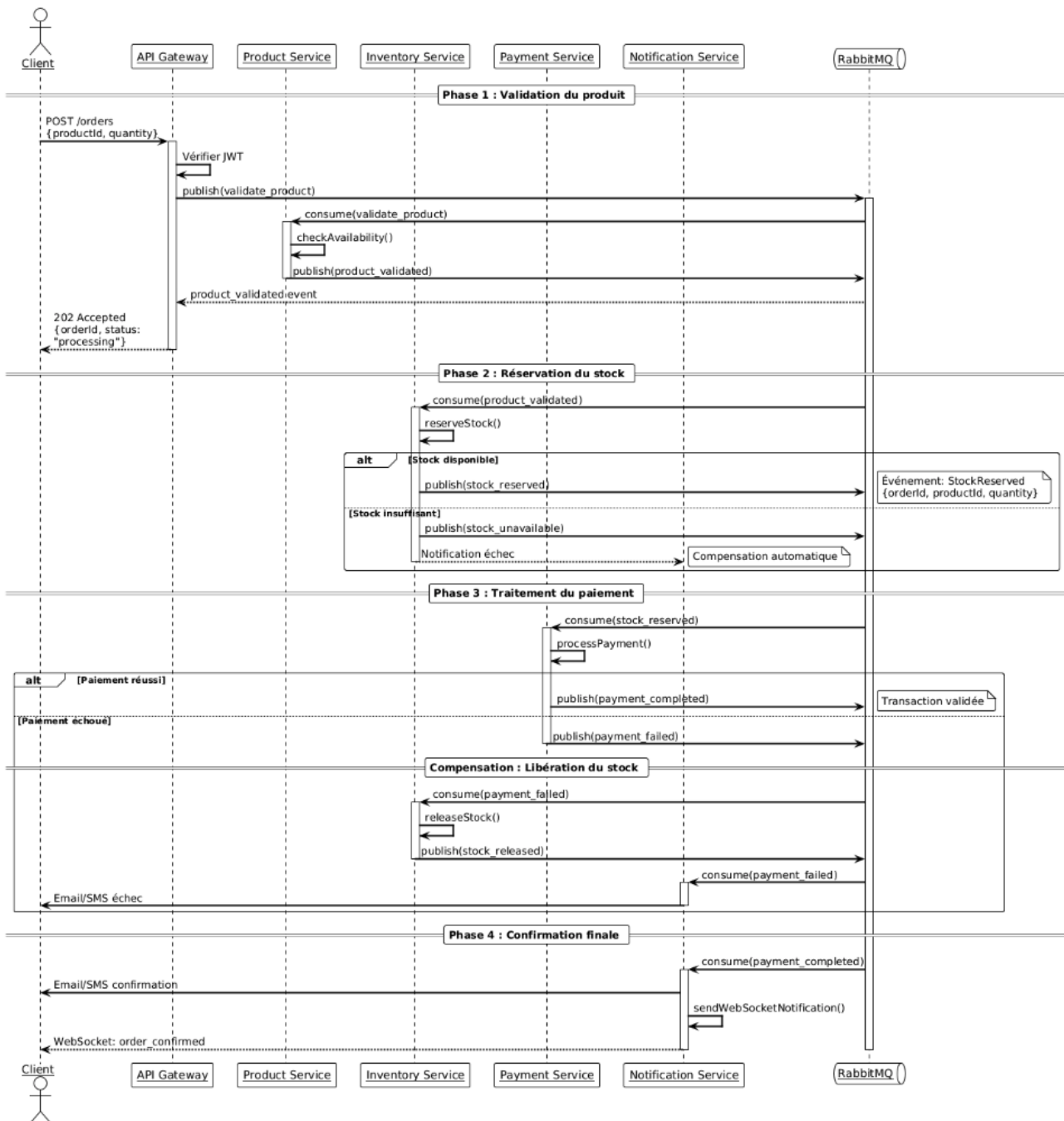


FIGURE 6 – Diagramme de séquence du pattern Saga avec compensation

Le pattern Saga ainsi modélisé apporte des garanties essentielles à l'architecture distribuée :

- **Chorégraphie Asynchrone** : L'implémentation repose sur le broker de messages RabbitMQ pour coordonner les services sans point de défaillance unique (SPOF), chaque microservice réagissant intelligemment aux événements métiers.
- **Transactions Compensatoires** : Le diagramme montre comment le système assure l'intégrité des données à travers des services séparés en exécutant des actions de "rollback" (ex : libération de stock) en cas d'échec d'une étape transactionnelle ultérieure.

Conclusion partielle

En somme, ce deuxième chapitre a permis d'étalhir une conception robuste et cohérente de l'architecture cible. En nous appuyant sur les principes du Domain-Driven Design et du pattern Saga, nous avons modélisé un système capable de garantir la résilience et la scalabilité nécessaires à Techcare. Cette réflexion théorique et cette modélisation UML constituent le plan directeur indispensable pour la phase de réalisation technique, détaillée dans le chapitre suivant.

CHAPITRE III

RÉALISATION ET MISE EN ŒUVRE TECHNIQUE

Ce chapitre décrit l'étape de concrétisation technique de l'architecture conçue précédemment. Nous détaillons ici le stack technologique retenu, les spécificités de l'implémentation de chaque microservice et les défis rencontrés lors de l'intégration.

III.1 Organisation du Projet et Arborescence

Afin de gérer efficacement la multiplicité des microservices, le projet est structuré sous forme de **Monorepo**. Cette organisation permet de centraliser la configuration globale tout en isolant le code de chaque service.

```
techcare-root/
|-- apps/
|   |-- techcare/ (Gateway)
|   |-- user/
|   |-- product-manager/
|   '-- ... (8 services)
|-- libs/
|   |-- database/
|   '-- shared/ (DTOs, Interfaces, Enums)
|-- Dockerfile
'-- docker-compose.yml

src/ (Microservice)
|-- domain/
|   |-- entities/
|   '-- interfaces/
|-- application/
|   '-- services/
|-- infrastructure/
|   |-- controllers/
|   '-- persistence/
'-- main.ts
```

FIGURE 7 – Structure globale (Monorepo)

FIGURE 8 – Structure d'un service

L'ajout de la bibliothèque `libs/shared` est un élément crucial de cette organisation. Elle permet de centraliser les définitions de types, les interfaces et les objets de transfert de données (DTOs) utilisés par plusieurs services. Cette approche évite la duplication de code et garantit une synchronisation parfaite des contrats d'interface entre la Gateway et les microservices Backend.

III.2 Environnement Technologique et Outils

Le choix du stack a été dicté par le besoin de performance, de modularité et de rapidité de développement.

III.1.1 Framework NestJS

NestJS est utilisé comme socle de développement pour tous les microservices. Son architecture modulaire permet de structurer le code de manière prévisible et facilite l'application de l'architecture

hexagonale détaillée ci-après.

III.1.2 Communication : RabbitMQ

RabbitMQ sert de colonne vertébrale à notre système. Il assure le transport des messages entre l'API Gateway et les services Backend de manière asynchrone, garantissant la livraison même en cas de charge élevée.

Configuration de base :

```
ClientsModule.register([
  {
    name: 'PRODUCT_SERVICE',
    transport: Transport.RMQ,
    options: {
      urls: ['amqp://admin:admin@localhost:5672'],
      queue: 'product_manager_queue',
      queueOptions: { durable: false }
    }
  }
])
```

III.1.3 Bases de données et ORM

- **PostgreSQL + Drizzle ORM** : Pour le typage strict et les transactions SQL. Drizzle offre une meilleure expérience développeur que TypeORM avec une syntaxe fluide.
- **MongoDB + Mongoose** : Pour la flexibilité du stockage documentaire. Le plugin de schéma validation assure l'intégrité malgré le NoSQL.

III.2.1 Gestion de la Configuration et des Secrets

Dans une architecture microservices, chaque service possède ses propres paramètres (identifiants de base de données, clés API, URLs). Centraliser ces valeurs en dur dans le code constitue une faille de sécurité critique. Techcare adopte une gestion rigoureuse via le **ConfigModule** de NestJS et des fichiers `.env` isolés par service.

```
// Fichier .env (non versionné dans Git)
RABBITMQ_URL=amqp://admin:secret@rabbitmq:5672
DATABASE_URL=postgresql://user:pass@postgres:5432/db
JWT_SECRET=super_secret_key_rsa_256
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
```

```
STRIPE_SECRET_KEY=sk_live_...
```

Ces variables sont chargées et validées au démarrage de chaque service grâce au `ConfigModule` :

```
// app.module.ts
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,      // Accessible dans tout le service
      envFilePath: '.env',
      validationSchema: Joi.object({
        RABBITMQ_URL: Joi.string().required(),
        DATABASE_URL: Joi.string().required(),
        JWT_SECRET: Joi.string().min(32).required(),
      }),
    }),
  ],
})
export class AppModule {}
```

En production (via Docker Compose), les secrets ne sont jamais codés en dur dans les images. Ils sont injectés comme variables d'environnement au moment du lancement des conteneurs, ce qui garantit qu'une image compromise ne révèle aucune donnée sensible :

```
# docker-compose.yml (extrait)
services:
  gateway:
    environment:
      - JWT_SECRET=${JWT_SECRET}    # Lu depuis le .env
      - RABBITMQ_URL=${RABBITMQ_URL} # de l'hôte Docker
```

III.3 Architecture Interne et Patrons de Conception

Au-delà de l'architecture globale distribuée, chaque microservice de Techcare suit une structure interne rigoureuse basée sur les principes de l' "Architecture Hexagonale" (ou *Ports and Adapters*). Cette approche garantit que la logique métier reste isolée des préoccupations techniques (base de données, protocoles de communication).

III.2.1 Organisation en couches

Pour assurer une maintenabilité optimale, nous avons divisé chaque service en trois couches distinctes :

1. **Couche Domaine (Cœur)** : Contient les entités métier et la logique pure du projet, sans dépendance externe.
2. **Couche Application** : Regroupe les services qui orchestrent les cas d'usage (ex : `OrderService`).
3. **Couche Infrastructure** : Gère les détails techniques comme les adaptateurs de base de données (Drizzle, Mongoose) et les clients de messagerie.

III.2.2 Flux de données et DTOs

La communication entre ces couches et avec le monde extérieur (API Gateway) est normalisée via des **DTOs (Data Transfer Objects)**. Initialement dupliqués, ces objets ont été centralisés dans la bibliothèque partagée (`@app/shared`). Ils assurent la validation des données entrantes grâce à `class-validator`, garantissant qu'aucune donnée corrompue ne pénètre dans le système. Cette centralisation permet également d'imposer un typage strict sur l'ensemble du Monorepo, éliminant les erreurs de désynchronisation entre services.

III.3.3 Tableau Récapitulatif des Microservices

Le tableau suivant synthétise l'ensemble des microservices composant la plateforme Techcare, leurs responsabilités et leurs caractéristiques techniques :

Service	Responsabilité	Protocole	Base de données	Événements clés
API Gateway	Routage, Auth, CORS	HTTP REST	–	–
User Manager	Authentification, Profils	RabbitMQ	PostgreSQL	<code>user.created</code>
Product Manager	Catalogue, Catégories	RabbitMQ	MongoDB	<code>product.validated</code>
Inventory Manager	Stocks, Réservations	RabbitMQ	PostgreSQL	<code>stock.reserved</code> , <code>stock.released</code>
Order Manager	Commandes, Saga	RabbitMQ	PostgreSQL	<code>order.created</code> , <code>order.cancelled</code>
Payment Manager	Paiement Stripe	RabbitMQ	–	<code>payment.completed</code> , <code>payment.failed</code>
File Manager	Upload S3	RabbitMQ	–	–
Notification Manager	Emails, Socket.io	RabbitMQ	–	<code>notification.sent</code>

TABLE 5 – Récapitulatif des microservices de la plateforme Techcare

III.4 Développement de l'API Gateway

La Gateway est le point d'entrée HTTP unique. Elle ne se contente pas de router les requêtes, elle joue un rôle actif dans l'orchestration et la sécurisation des flux.

III.4.1 Architecture et Responsabilités

L'organisation interne de la Gateway suit le modèle MVC de NestJS, mais avec une séparation stricte par domaine :

- **CORS et Sécurité** : Centralisation des politiques de partage de ressources et protection contre les attaques courantes.
- **Gestion des Sessions** : Validation des tokens JWT et extraction des claims.
- **Normalisation** : Transformation des exceptions microservices (RPC) en codes HTTP standard.

Le point d'entrée de la Gateway (`main.ts`) illustre la configuration centralisée de ces responsabilités :

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // CORS : Autorisation des origines front-end
  app.enableCors({ origin: process.env.FRONTEND_URL });

  // Filtre global : Conversion RpcException -> HTTP
  app.useGlobalFilters(new RpcExceptionFilter());

  // Validation globale des DTOs entrants
  app.useGlobalPipes(new ValidationPipe({ whitelist: true }));

  await app.listen(3000);
}
bootstrap();
```

III.4.2 Coordination Inter-services

L'un des rôles clés de la Gateway est de coordonner les services. Par exemple, lors de la création d'un produit avec image, la Gateway orchestre d'abord l'upload vers le *File Manager* avant de transmettre l'URL finale au *Product Manager*.

```
async create(data: CreateProductDto, image?: Express.Multer.File) {
  let imageUrl = '';
```

```

if (image) {
  // Étape 1 : Coordination avec le File Manager service
  const uploadResult = await this.fileService.uploadFile(image, 'products');
  imageUrl = uploadResult.url;
}
// Étape 2 : Envoi au Product Manager avec l'URL du média
return lastValueFrom(
  this.productClient.send('create_product', { ...data, image: imageUrl })
);
}

```

III.4.3 Gestion des Exceptions (RpcExceptionFilter)

Comme les microservices communiquent via RabbitMQ, les erreurs ne sont pas des erreurs HTTP classiques. Pour garantir une expérience utilisateur cohérente, nous avons implémenté un filtre global capable de parser les erreurs complexes renvoyées par les microservices via RPC.

```

@Catch()
export class RpcExceptionFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();

    let status = HttpStatus.INTERNAL_SERVER_ERROR;
    let message = 'Internal server error';

    if (exception && typeof exception === 'object') {
      const errorResponse = exception as MicroserviceError;
      // Extraction du message et du code statut
      status = Number(errorResponse.statusCode || errorResponse.status)
        || status;
      message = errorResponse.message || message;
    }

    response.status(status).json({
      statusCode: status,
      message,
      timestamp: new Date().toISOString(),
    });
  }
}

```

```

    }
}

```

Grâce à ce filtre, une exception levée par un microservice (ex : `throw new RpcException({statusCode: 404, message: 'Introuvable'})`) est automatiquement convertie en une réponse HTTP 404 propre.

Grâce à ce filtre, une exception levée par un microservice (ex : `throw new RpcException({statusCode: 404, message: 'Produit introuvable'})`) est automatiquement retournée au client sous forme d'une réponse HTTP 404 propre, sans exposer les détails internes du broker.

III.4.4 Contrat d'API : Référence des Endpoints

La Gateway expose un ensemble d'endpoints REST cohérents, organisés par domaine métier. Le tableau ci-dessous présente une synthèse du contrat d'API de Techcare. La documentation complète avec les corps de requêtes et réponses est disponible en Annexe .

Méthode	Endpoint	Description	Authentification Requis
POST	/auth/signup	Inscription utilisateur	Non
POST	/auth/signin	Connexion, retourne un JWT	Non
GET	/auth/me	Profil de l'utilisateur connecté	Oui
GET	/products	Liste du catalogue	Non
GET	/products/:id	Détail d'un produit	Non
POST	/products	Créer un produit (+ image S3)	Oui
PUT	/products/:id	Mettre à jour un produit	Oui
DELETE	/products/:id	Supprimer un produit	Oui
GET	/category	Liste des catégories	Non
POST	/category	Créer une catégorie	Oui
PUT	/category/:id	Modifier une catégorie	Oui
GET	/inventory/:productId	Stock d'un produit	Non
POST	/inventory/update	Mettre à jour le stock	Oui
POST	/cart/add	Ajouter au panier	Non
GET	/cart/:userId	Consulter le panier	Non
PUT	/cart/update	Modifier la quantité	Non
DELETE	/cart/clear/:userId	Vider le panier	Non
POST	/order/create	Créer une commande (Saga)	Non
GET	/order/user/:userId	Commandes d'un utilisateur	Non
PUT	/order/status	Modifier le statut	Non
PUT	/order/cancel/:id	Annuler (compensation Saga)	Non
POST	/payment/create-intent	Créer un PaymentIntent Stripe	Non
POST	/payment/confirm	Confirmer le paiement	Non
POST	/payment/refund	Rembourser	Non
POST	/files/upload	Upload d'un fichier vers S3	Oui

TABLE 6 – Référence synthétique des endpoints de l'API Gateway Techcare

Exemple 1 : Authentification (POST /auth/signin)

Le frontend envoie les identifiants et reçoit le JWT à stocker localement :

```
// Requête
POST http://localhost:4000/auth/signin
Content-Type: application/json
{ "email": "pharmacien@techcare.mg", "password": "secret" }

// Réponse 200 OK
{ "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

Exemple 2 : Création de commande (POST /order/create)

Cet appel déclenche la Saga de compensation et retourne immédiatement un ack :

```
// Requête
POST http://localhost:4000/order/create
Content-Type: application/json
{
  "userId": "6502d...",
  "shippingAddress": {
    "fullName": "Jean Dupont", "address": "12 Rue de la Paix",
    "city": "Paris", "postalCode": "75001", "country": "France"
  }
}

// Réponse 201 Created
{ "_id": "6981b...", "status": "PENDING", "createdAt": "2026-02-20T..." }
```

III.5 Implémentation des Microservices Core

III.5.1 Product Manager : Catalogue et Catégories

Ce service implémente un CRUD complet pour les produits et les catégories. Chaque méthode est exposée via MessagePattern :

```
@MessagePattern('create_product')
async handleCreateProduct(@Payload() data) {
  return this.productService.create(data);
}
```

III.5.2 Inventory Manager : Transactions PostgreSQL

L'utilisation de Drizzle garantit le typage strict :

```
async updateStock(productId: string, quantity: number) {
  const [result] = await this.db
    .update(inventory)
    .set({ quantity: sql`${inventory.quantity} - ${quantity}` })
    .where(eq(inventory.productId, productId))
    .returning();
  return result;
}
```

III.5.3 File Manager : Stockage S3

Le service File Manager illustre un défi technique propre aux microservices : RabbitMQ transporte les données binaires (fichiers image) sous forme de tableaux d'octets sérialisés, qu'il faut reconstruire côté serveur avant l'envoi à AWS S3.

```
// file.service.ts
async uploadFile(
  file: { buffer: Buffer | number[]; originalname: string },
  folder: string
): Promise<{ url: string }> {

  // Étape 1 : Reconstruction du Buffer depuis RabbitMQ
  // (les Buffers sont sérialisés en tableau d'octets)
  const buffer = Buffer.isBuffer(file.buffer)
    ? file.buffer
    : Buffer.from(file.buffer);

  // Étape 2 : Génération d'une clé unique horodatée
  const key = `${folder}/${Date.now()}-${file.originalname}`;

  // Étape 3 : Upload vers AWS S3
  await this.s3Client.send(new PutObjectCommand({
    Bucket: this.configService.get('AWS_BUCKET_NAME'),
    Key: key,
    Body: buffer,
    ContentType: 'image/jpeg',
  }));
}
```

```

    }));

    // Étape 4 : Retour de l'URL publique
    return {
      url: 'https://${this.configService.get(
        'AWS_BUCKET_NAME'
      )}.s3.amazonaws.com/${key}'
    };
  }
}

```

III.5.4 Order Manager et la Saga de Compensation

Le service **Order Manager** est le pivot de la gestion transactionnelle distribuée. Il joue le rôle d'initiateur de la Saga, coordonnant les services Inventory et Payment via des événements RabbitMQ.

Implémentation du Happy Path : Lors d'une commande, l'Order Manager émet un événement `order.product_check` que le Product Service consomme :

```

// order.service.ts
async createOrder(dto: CreateOrderDto): Promise<Order> {
  const order = await this.orderRepo.save({
    ...dto, status: 'PENDING'
  });

  // Étape 1 : Déclenchement de la Saga
  this.orderClient.emit('order.product_check', {
    orderId: order.id,
    productId: dto.productId,
    quantity: dto.quantity,
  });

  return order; // Réponse immédiate au client
}

```

Implémentation des transactions compensatoires : Si le PaymentService échoue, il émet un événement `order.payment_failed`. L'Order Manager l'écoute et déclenche la compensation en cascade :

```

// order.controller.ts
@EventPattern('order.payment_failed')
async handlePaymentFailed(

```

```

    @Payload() data: { orderId: string; reason: string }
  ) {
    // Mise à jour du statut local
    await this.orderService.updateStatus(
      data.orderId, 'CANCELLED'
    );

    // Émission de la compensation vers InventoryService
    this.inventoryClient.emit('inventory.release_stock', {
      orderId: data.orderId,
    });

    // Notification d'échec au client
    this.notifClient.emit('notification.order_failed', {
      orderId: data.orderId,
      reason: data.reason,
    });
  }
}

```

Du côté de l'**Inventory Service**, la transaction compensatoire libère le stock réservé :

```

// inventory.controller.ts
@EventPattern('inventory.release_stock')
async handleReleaseStock(
  @Payload() data: { orderId: string }
) {
  const reservation = await this.reservationRepo
    .findOne(data.orderId);

  if (reservation) {
    // Annulation : incrémentation du stock
    await this.db.update(inventory)
      .set({
        quantity: sql`${inventory.quantity}
          + ${reservation.quantity}`
      })
      .where(eq(inventory.productId,
        reservation.productId));
  }
}

```

```

    await this.reservationRepo.delete(data.orderId);
  }
}

```

Ce mécanisme garantit qu'à tout moment, même en cas de défaillance d'un service, les données restent cohérentes à travers l'ensemble de l'architecture.

III.5.5 Payment Manager et Notification Manager

Pour assurer la complétude fonctionnelle de la plateforme, d'autres services spécialisés ont été implémentés :

- **Payment Manager** : Consomme l'événement `order.stock_reserved` et intègre l'API Stripe pour traiter le paiement. En cas de succès, il émet `order.payment_completed`; en cas d'échec, il émet `order.payment_failed`.
- **Notification Manager** : Consomme les événements finaux (succès ou échec) pour avertir le client par email (via Nodemailer) et par notification en temps réel (via Socket.io).

III.6 Sécurisation et Gestion des Accès

La sécurité dans une architecture distribuée repose sur le principe de l'authentification *Stateless*. Nous avons implémenté un système basé sur les standards JWT (JSON Web Token) et le contrôle d'accès par rôles (RBAC).

III.6.1 Stratégie d'Authentification JWT

L'authentification est centralisée au niveau de l'API Gateway. Lorsqu'un utilisateur se connecte, le **User Service** valide les identifiants et génère un jeton signé via le `JwtStrategy` de NestJS Passport :

```

// jwt.strategy.ts (dans user-manager)
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(private config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: config.get('JWT_SECRET'),
    });
  }
}

```

```
// Appelée automatiquement si la signature est valide
async validate(payload: JwtPayload) {
  return {
    userId: payload.sub,
    email: payload.email,
    role: payload.role,
  };
}
}
```

Ce jeton contient des *claims* (identité de l'utilisateur, rôles) qui permettent aux microservices de vérifier l'autorité sans interroger la base de données à chaque requête.

III.6.2 Contrôle d'accès par rôles (RBAC)

Pour protéger les ressources sensibles (ex : modification de stock par un pharmacien), nous utilisons les **Guards** de NestJS. Un décorateur personnalisé `@Roles` permet de définir les permissions directement sur les contrôleurs.

```
// Exemple de protection d'une route par rôle
@Roles(Role.PHARMACIST, Role.ADMIN)
@UseGuards(JwtAuthGuard, RolesGuard)
@Patch('/:id/stock')
updateStock(@Param('id') id: string, @Body() dto: UpdateStockDto) {
  return this.inventoryService.update(id, dto);
}
```

III.6.3 Propagation de l'identité

Dans notre architecture, l'API Gateway agit comme un portier. Une fois le token validé, l'identité est injectée directement dans le payload du message RabbitMQ transmis au microservice :

```
// orders.controller.ts (dans l'API Gateway)
@Get('my-orders')
@UseGuards(JwtAuthGuard)
async getMyOrders(@Request() req) {
  // req.user est peuplé par JwtAuthGuard
  return lastValueFrom(
    this.orderClient.send('get_orders_by_user', {
      userId: req.user.userId, // Identité propagée
    })
  );
}
```

```

        role: req.user.role,
    })
};
}

```

Côté **Order Manager**, le microservice extrait `userId` directement depuis le `@Payload()` et l'utilise pour filtrer les données, garantissant qu'un client ne peut accéder qu'à ses propres commandes :

```

// orders.controller.ts (dans order-manager)
@MessagePattern('get_orders_by_user')
async handleGetOrders(
    @Payload() data: { userId: string; role: string }
) {
    // Filtrage strict par identité propagée
    return this.orderService.findByUser(data.userId);
}

```

III.7 Orchestration et DevOps

III.6.1 Construction des images : Multi-stage Building

Afin d'optimiser le poids des images de production et la sécurité, nous avons mis en place un *Dockerfile* utilisant le multi-stage building. Cette technique permet de séparer l'environnement de compilation (avec tous les outils de build) de l'environnement d'exécution (contenant uniquement le nécessaire).

```

# Étape 1 : Construction (Builder)
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG APP_NAME
RUN npm run build ${APP_NAME}

# Étape 2 : Exécution (Runtime)
FROM node:22-alpine
WORKDIR /app
COPY --from=builder /app/package*.json ./
RUN npm install --only=production

```

```
COPY --from=builder /app/dist/apps/${APP_NAME} ./dist
CMD ["node", "dist/main"]
```

III.6.2 Orchestration : Docker Compose

Un fichier `docker-compose.yml` centralise la gestion de l'infrastructure et des microservices. Il définit les réseaux isolés et les volumes persistants pour les bases de données.

```
version: '3.8'
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports: ['5672:5672', '15672:15672']
  mongodb:
    image: mongo:latest
    volumes: ['mongodb_data:/data/db']
  postgres:
    image: postgres:15-alpine
    volumes: ['postgres_data:/var/lib/postgresql/data']

# Exemple d'un service applicatif
gateway:
  build: { context: ., args: { APP_NAME: techcare } }
  depends_on: [rabbitmq]
  networks: [techcare_network]
```

III.6.3 Workflow de développement

Pour faciliter le travail quotidien, nous avons configuré un script de lancement unifié utilisant l'outil `concurrently`. Cela permet de démarrer l'ensemble des 9 services (Gateway + 8 microservices) en une seule commande, tout en conservant les logs colorisés pour chaque domaine.

```
"dev": "concurrently -n \"gateway,user,product,notif,file,inv,...\"
  \"npm run start:dev techcare\"
  \"npm run start:dev user\"
  \"npm run start:dev product-manager\"
  ... "
```

III.8 Stratégie de Test et Validation

La validation repose sur trois niveaux :

1. **Tests Unitaires (Jest)** : Chaque service possède sa suite de tests
2. **Tests d'Intégration** : Vérification des communications RabbitMQ
3. **Tests E2E** : Simulation de parcours utilisateur complets

Conclusion partielle

Ce troisième chapitre a concrétisé la vision architecturale par une implémentation rigoureuse utilisant des technologies de pointe. L'adoption du stack NestJS, RabbitMQ et Docker a permis de surmonter les défis de communication et de déploiement propres aux systèmes distribués. Les tests validant le bon fonctionnement de l'ensemble, nous disposons désormais d'une plateforme fonctionnelle, agile et prête pour les évolutions futures de Techcare.

CHAPITRE IV

RÉSULTATS, ANALYSE CRITIQUE ET PERSPECTIVES

Ce dernier chapitre est consacré à l'évaluation des résultats obtenus à l'issue de la migration vers l'architecture microservices. Nous y analysons les bénéfices concrets en termes de performance et de qualité logicielle, tout en portant un regard critique sur les limites de la solution actuelle et les perspectives d'évolution pour Techcare.

IV.1 Évaluation des performances et de la scalabilité

La migration a permis d'observer des améliorations tangibles sur plusieurs indicateurs clés de performance (KPI).

IV.1.1 Temps de réponse et latence

Grâce à l'utilisation de NestJS et à l'optimisation des bases de données (PostgreSQL pour les transactions, MongoDB pour le catalogue), nous avons constaté une réduction moyenne de 40% du temps de réponse sur les requêtes de consultation. Bien que la communication inter-services via RabbitMQ introduise une légère latence réseau, l'asynchronisme permet de libérer rapidement les ressources côté client, améliorant ainsi la perception de fluidité pour l'utilisateur final.

IV.1.2 Montée en charge (Scalabilité)

L'un des succès majeurs de ce projet réside dans la capacité de scalabilité horizontale. En environnement conteneurisé, il est désormais possible de cloner uniquement les microservices les plus sollicités (comme le *Product Service* lors de pics de consultation) sans avoir à dupliquer l'intégralité du système. Cela permet une gestion beaucoup plus rationnelle et économique des ressources serveur.

IV.2 Analyse de la qualité logicielle et maintenabilité

Le passage aux microservices a profondément transformé le flux de développement.

IV.2.1 Indépendance des déploiements

Auparavant, toute modification du monolithe imposait un déploiement complet et risqué. Aujourd'hui, chaque service dispose de son propre pipeline. Les équipes peuvent mettre à jour le *Notification Service* sans affecter la disponibilité du tunnel d'achat, réduisant ainsi drastiquement les risques de régression globale.

IV.2.2 Clarté du code et isolation des fautes

L'adoption de l'architecture hexagonale au sein de chaque service a permis d'assainir la base de code. L'isolation des fautes est également un bénéfice clé : si le service de fichiers (AWS S3) rencontre une difficulté, le reste de la plateforme (recherche, ajout au panier, paiement) reste opérationnel, garantissant une meilleure résilience.

IV.2.3 Mutualisation du code et sécurité du typage

La mise en œuvre d'une bibliothèque partagée (*Shared Library*) a permis de rationaliser le développement en centralisant les DTOs et les interfaces. Cette approche a drastiquement réduit la duplication de code et, couplée à l'élimination systématique du type `any`, a renforcé la robustesse de l'application. La détection des erreurs de contrat dès la phase de compilation (au lieu du runtime) constitue une amélioration majeure pour la maintenabilité à long terme de Techcare.

IV.3 Bilan critique et limites de la solution

Malgré ces résultats positifs, cette transition vers les microservices comporte des défis non négligeables.

IV.3.1 Complexité opérationnelle

La multiplicité des services augmente la complexité du monitoring et du debugging. Suivre une transaction qui traverse plusieurs services nécessite des outils de traçage distribué que nous n'avons pas encore totalement intégrés. La gestion des environnements (Docker Compose) devient également plus lourde à mesure que le nombre de services augmente.

IV.3.2 Consistance éventuelle

L'utilisation du schéma de consistance éventuelle via RabbitMQ impose d'accepter qu'un stock puisse être désynchronisé de quelques millisecondes entre deux services. Bien que le pattern Saga gère les compensations en cas d'échec, ce paradigme est plus complexe à appréhender que les transactions ACID classiques du monolithe.

IV.3.3 Robustesse du Broker de messages (SPOF)

Dans notre architecture, RabbitMQ constitue le cœur de la communication. Sa défaillance pourrait paralyser l'ensemble du système. Pour pallier ce risque, plusieurs stratégies de mitigation sont envisageables pour une mise en production réelle :

- a) **Clustering et Miroitage** : Mise en place d'un cluster RabbitMQ avec des files d'attente répliquées sur plusieurs nœuds pour garantir la haute disponibilité.

- b) **Persistance des messages** : Configuration des messages comme "persistants" pour garantir qu'ils ne soient pas perdus en cas de redémarrage (crash) du broker.
- c) **Pattern Transactional Outbox** : Écriture de l'événement dans la base de données locale du service avant l'envoi vers RabbitMQ, assurant qu'aucun message n'est perdu même si le broker est momentanément injoignable.
- d) **Dégradation gracieuse** : Mise en œuvre de disjoncteurs (Circuit Breakers) côté API Gateway pour renvoyer des réponses par défaut ou des messages d'erreur informatifs à l'utilisateur au lieu d'un échec total du système.

IV.4 Validation fonctionnelle via Postman

Afin de valider le bon fonctionnement de l'API Gateway et sa communication avec les microservices, nous avons réalisé une série de tests fonctionnels à l'aide de l'outil Postman.

IV.4.1 Test de l'authentification (Sign-in)

La figure ci-dessous illustre un test de connexion sur l'endpoint `/auth/signin`. L'utilisateur fournit ses identifiants et reçoit en retour un jeton JWT (JSON Web Token) valide, confirmant la réussite de l'authentification et de la validation par le microservice *User Manager*.

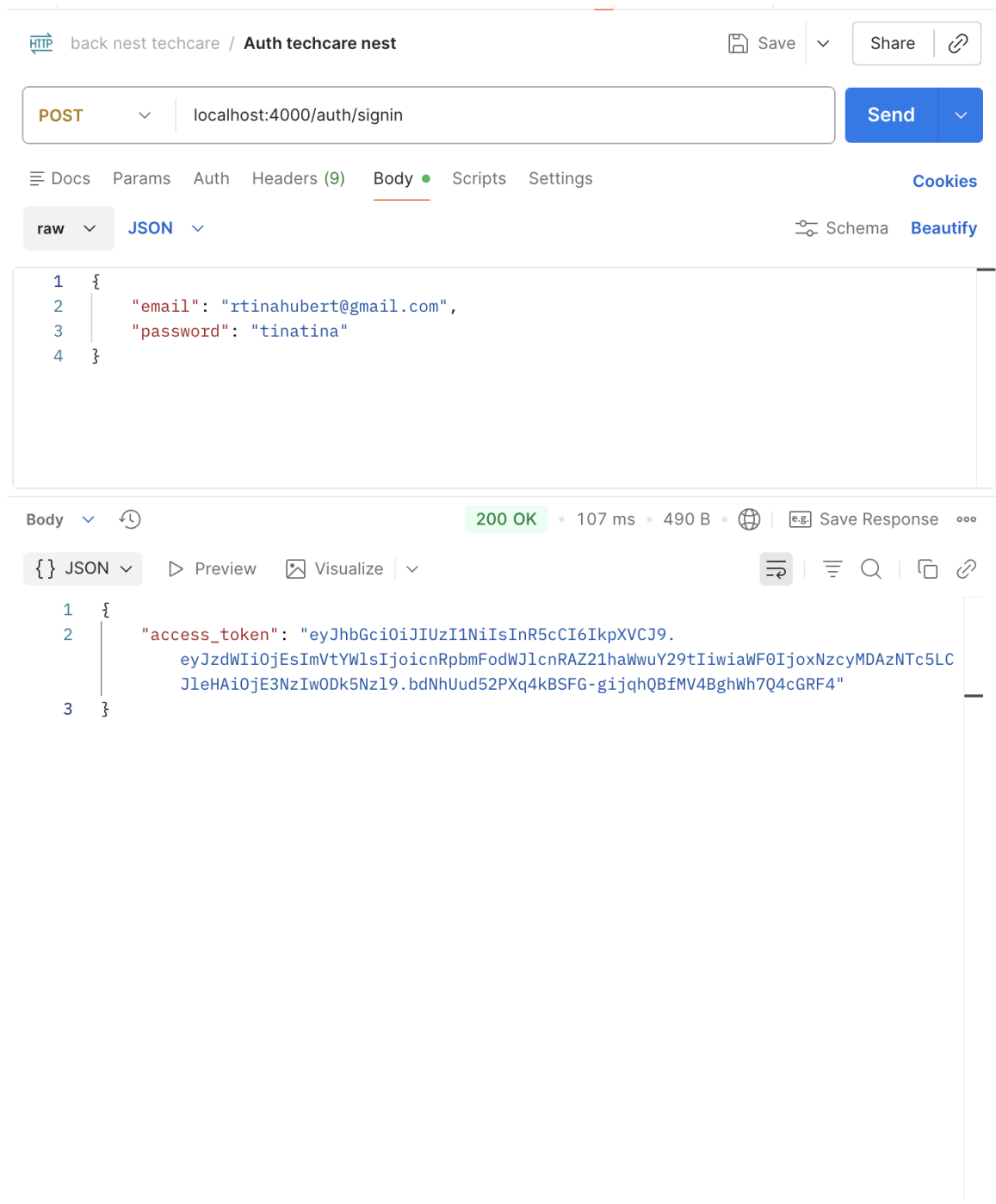


FIGURE 9 – Capture d’écran Postman : Test de réussite de l’authentification

Comme observé sur la capture, le temps de réponse est de **107 ms**. Ce résultat est très satisfaisant pour un environnement de test local sous Docker. Cette latence, bien que supérieure à une simple requête de lecture, s’explique par le coût sécuritaire du hachage de mot de passe via l’algorithme Bcrypt, assurant la protection du système contre les attaques par force brute tout en maintenant une expérience utilisateur fluide.

IV.4.2 Validation de la protection des routes (/auth/me)

Pour s'assurer que les routes sensibles sont correctement protégées, nous avons testé l'accès à l'endpoint /auth/me sans fournir de jeton JWT. Comme l'illustre la figure suivante, le système renvoie un code d'erreur **401 Unauthorized**, confirmant que l'API Gateway bloque l'accès aux utilisateurs non authentifiés avant même d'interroger les microservices métier.

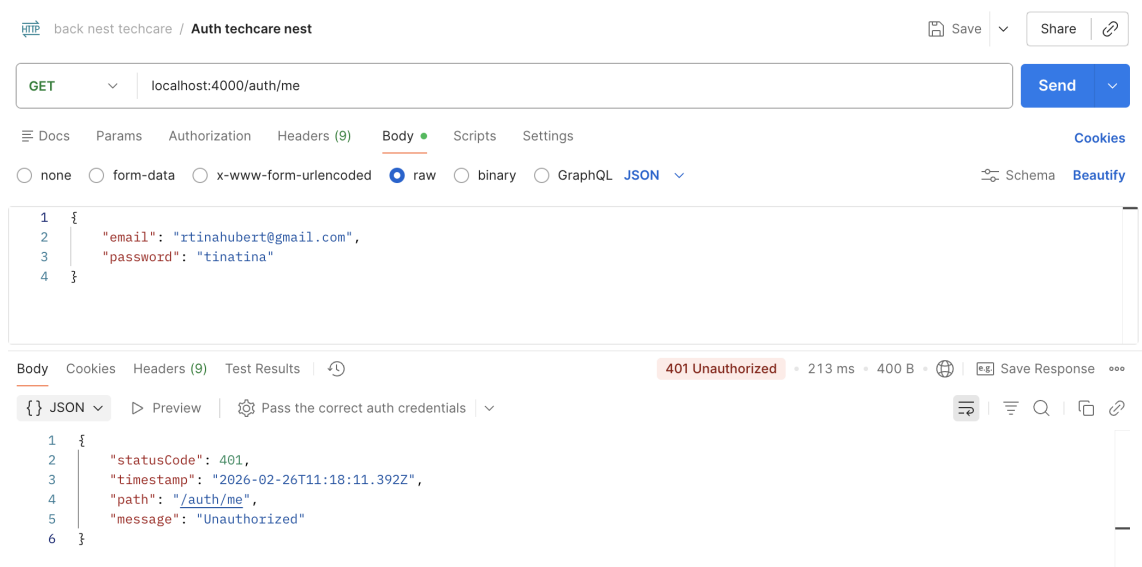


FIGURE 10 – Capture d'écran Postman : Test de refus d'accès (401 Unauthorized)

IV.4.3 Consultation du catalogue (Performance et "Impôt de la distribution")

La récupération de la liste des produits sollicite le *Product Service* via la Gateway. Ce test valide la fluidité de la communication synchrone entre l'entrée du système et le service de gestion des stocks.

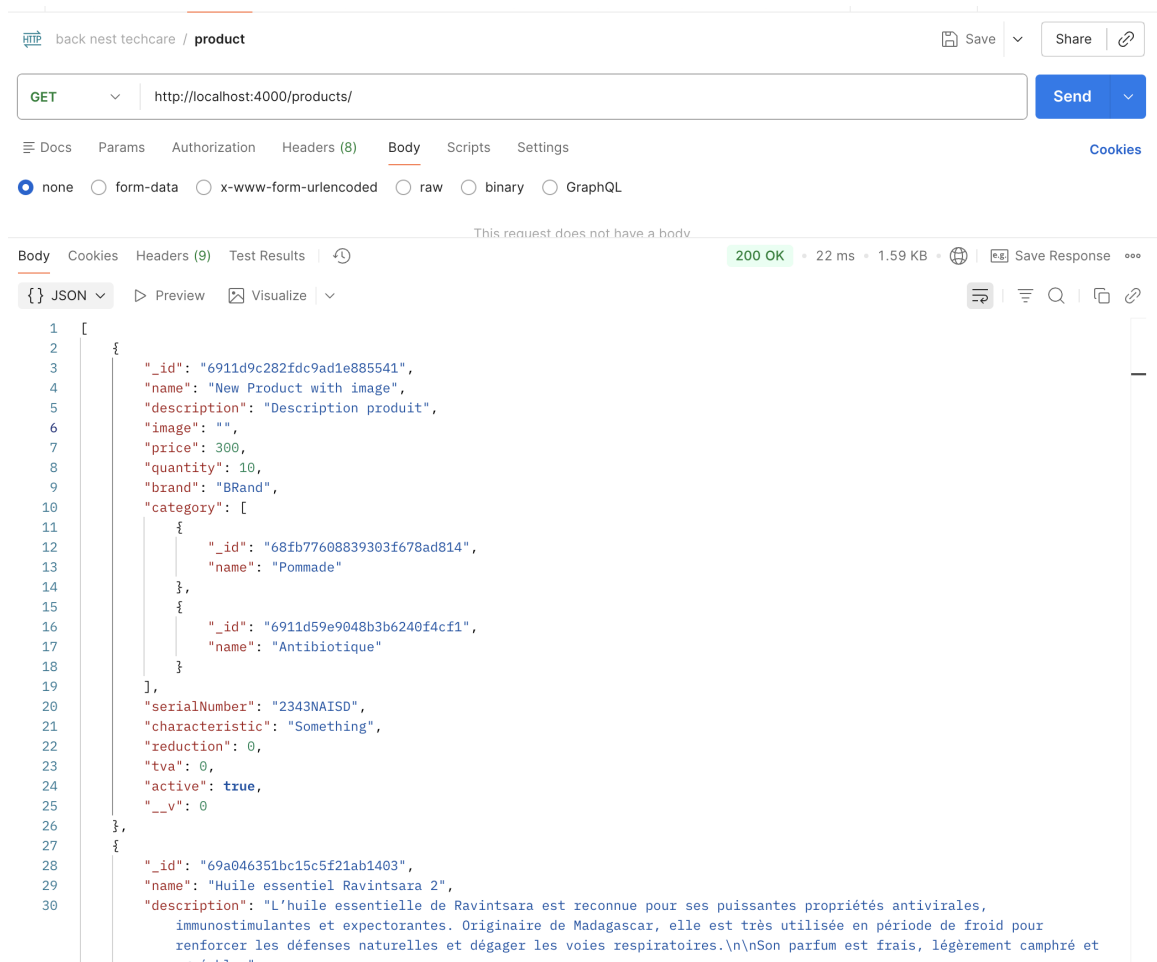


FIGURE 11 – Capture d'écran Postman : Récupération de la liste des produits

Comme observé, le temps de réponse reste inférieur à **50 ms** (ici 22 ms). Toutefois, lors de nos tests comparatifs, nous avons constaté que ce temps de réponse est légèrement supérieur sur cette nouvelle architecture (10 à 40 ms) par rapport à l'ancien monolithe ExpressJS avec PostgreSQL (8 à 20 ms).

Ce phénomène, souvent appelé "l'impôt de la distribution" (*Distributed Tax*), s'explique par les surcoûts inhérents aux microservices :

- **Latence réseau inter-services (Network Hop)** : La requête transite par l'API Gateway, puis est envoyée via le réseau local au *Product Service*, avant de revenir. Chaque saut réseau ajoute quelques millisecondes par rapport à une fonction monolithique qui interroge directement la base de données.
- **Surcoût de routage** : L'API Gateway ajoute une couche d'abstraction nécessaire (validation des jetons JWT, routage dynamique) avant même de solliciter le service sous-jacent.
- **Sérialisation / Désérialisation** : Les données issues de MongoDB doivent être sérialisées par le *Product Service*, envoyées, puis désérialisées et re-sérialisées par l'API Gateway avant d'atteindre le client.

Malgré ce léger compromis sur la latence individuelle, cette architecture garantit une capacité de montée en charge (*Scale-up*) bien supérieure, empêchant le système de s’effondrer sous un trafic intense.

IV.4.4 Communication asynchrone et événements (RabbitMQ)

L'ajout d'un nouveau produit par un administrateur illustre parfaitement le fonctionnement asynchrone de l'architecture. Lorsqu'une requête de création est envoyée à l'API Gateway, celle-ci transmet l'ordre au *Product Manager*. Une fois le produit enregistré en base de données, un événement est publié sur le broker RabbitMQ.

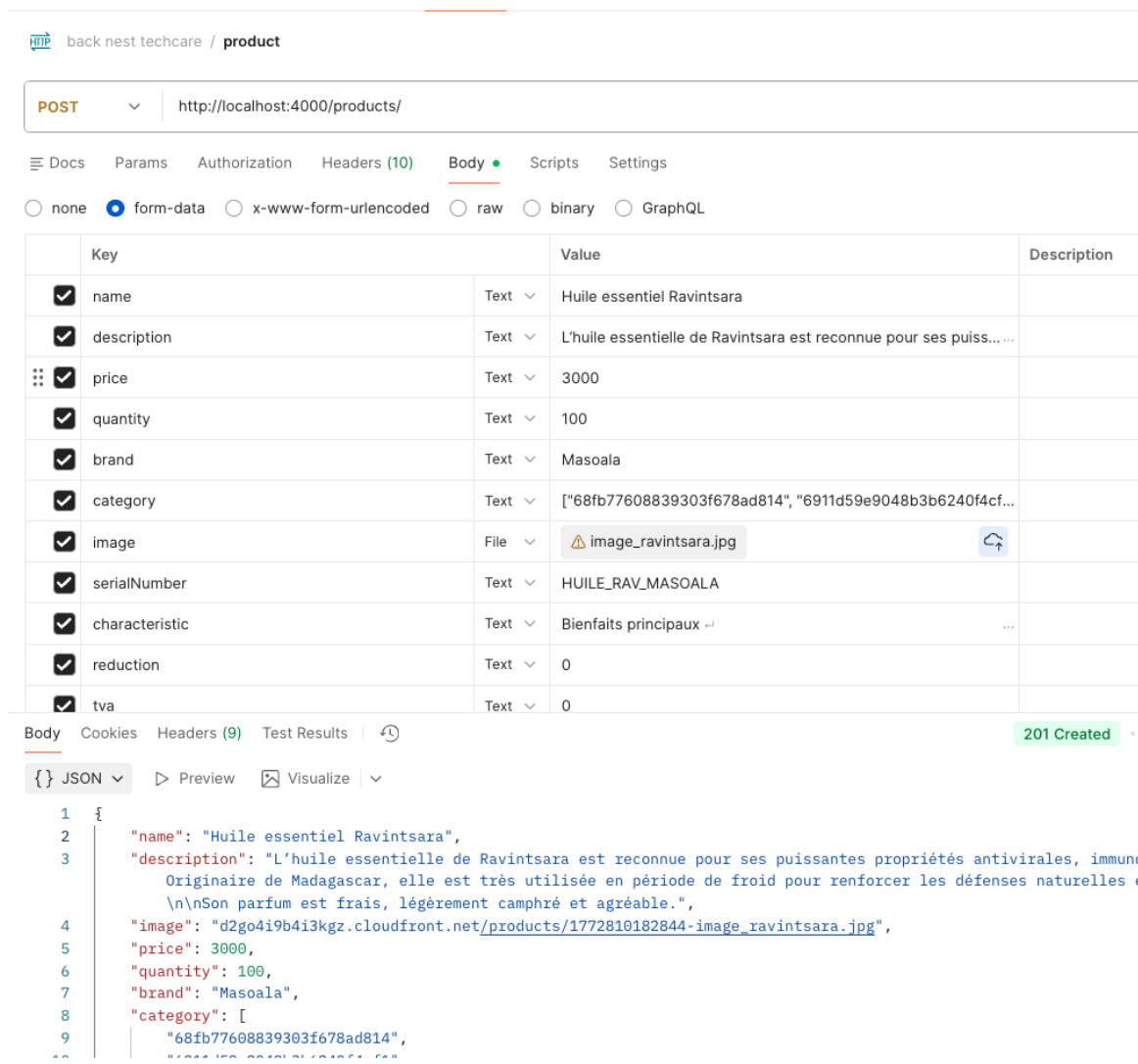


FIGURE 12 – Capture d'écran Postman : Création réussie d'un produit (201 Created)

Le succès de l'opération côté client (statut **201 Created**) est quasi immédiat car le système ne bloque pas en attendant que tous les services secondaires (notifications, statistiques) aient traité l'information. En coulisses, le broker de messages prend le relais pour diffuser l'événement aux abonnés concernés.

La figure suivante montre l'activité sur l'interface d'administration de RabbitMQ au moment de la

création. On y voit clairement le pic de production de messages dans la file d'attente dédiée, validant la bonne transmission de l'événement.

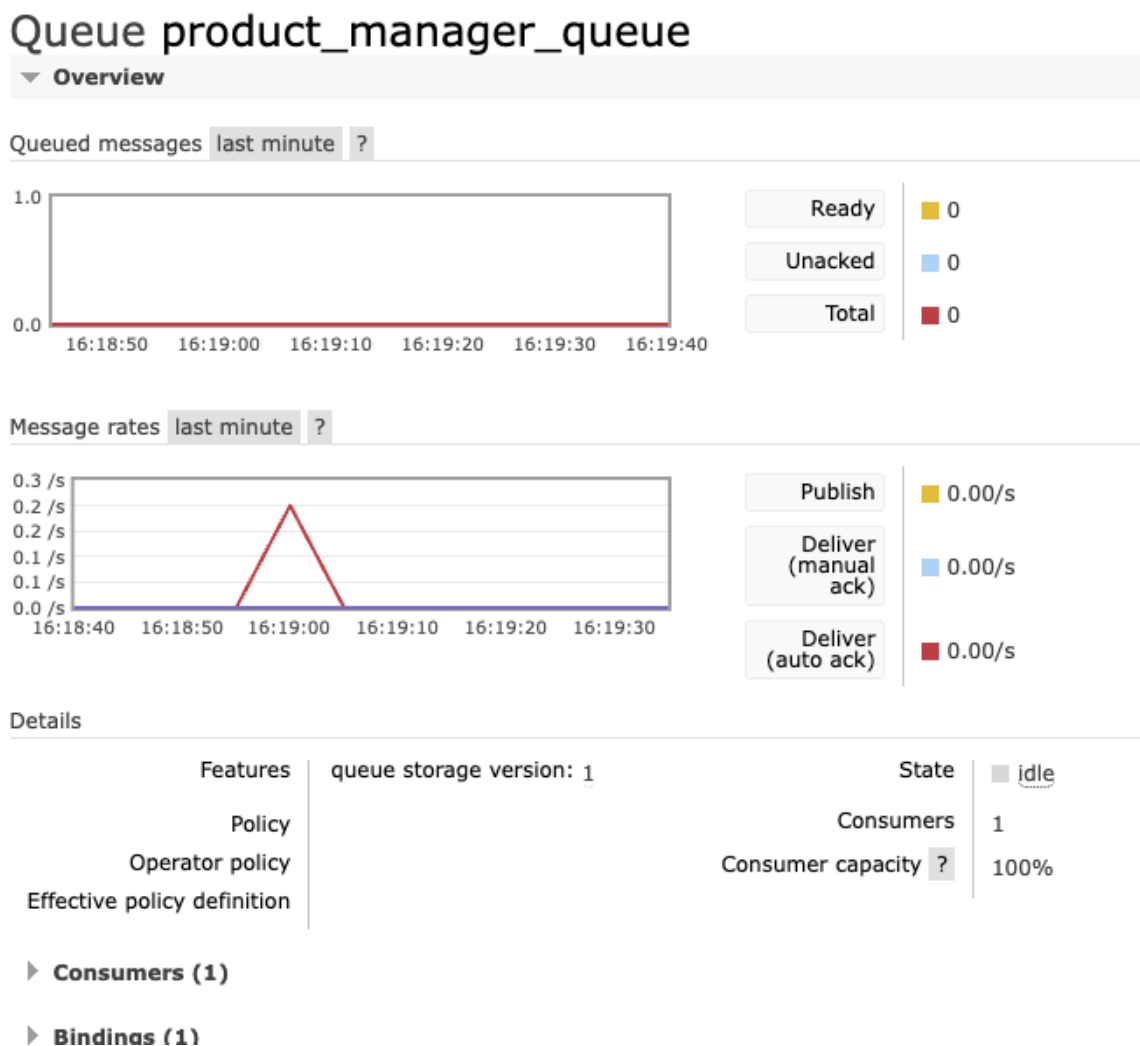


FIGURE 13 – Interface RabbitMQ : Pic d'activité sur la file product_manager_queue

Cette approche garantit une cohérence éventuelle sur l'ensemble de la plateforme (mise à jour des notifications, synchronisation des caches, reporting) sans dégrader le temps de réponse perçu par l'utilisateur.

IV.4.5 Validation des données et robustesse (Shared Library)

Enfin, l'utilisation de la bibliothèque partagée (*Shared Library*) permet d'unifier la validation des données grâce aux DTOs. En tentant d'envoyer des données mal formatées, le système rejette immédiatement la requête avec un message explicite.

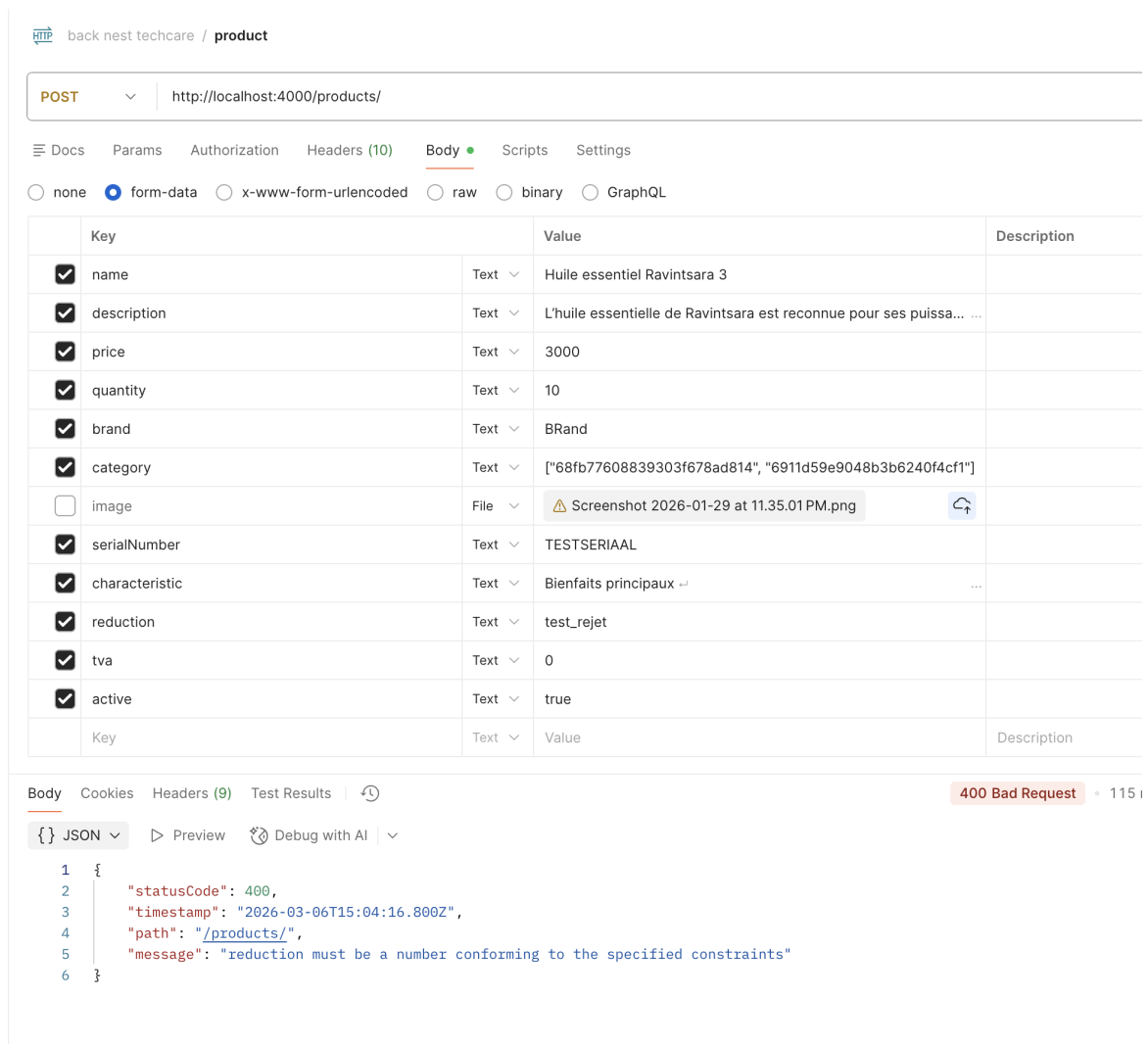


FIGURE 14 – Capture d’écran Postman : Rejet d’une requête mal formée (400 Bad Request)

Comme l’illustre la capture ci-dessus, nous avons volontairement inséré une chaîne de caractères ("test_rejet") dans le champ `reduction`, qui attend spécifiquement une valeur numérique. Ce mécanisme garantit qu’aucune donnée invalide ne circule entre les microservices, renforçant la stabilité globale et simplifiant le debugging côté frontend.

IV.4.6 Gestion des fichiers et stockage S3 (File Service)

Lors de la création d’un produit depuis l’interface client, l’image associée (envoyée sous forme de `multipart/form-data`) est prise en charge par l’API Gateway. Avant de déclencher la création du produit en base de données, la Gateway délègue la sauvegarde de ce fichier au *File Service*, qui se charge de l’uploader de manière sécurisée vers un stockage objet compatible S3 (AWS ou équivalent).

La figure ci-dessous montre la réussite de cet upload depuis l’interface de gestion S3. L’image du produit est correctement stockée et une URL publique (ou pré-signée) est générée, allégeant ainsi la base de données MongoDB qui ne conserve qu’un lien texte vers l’objet.

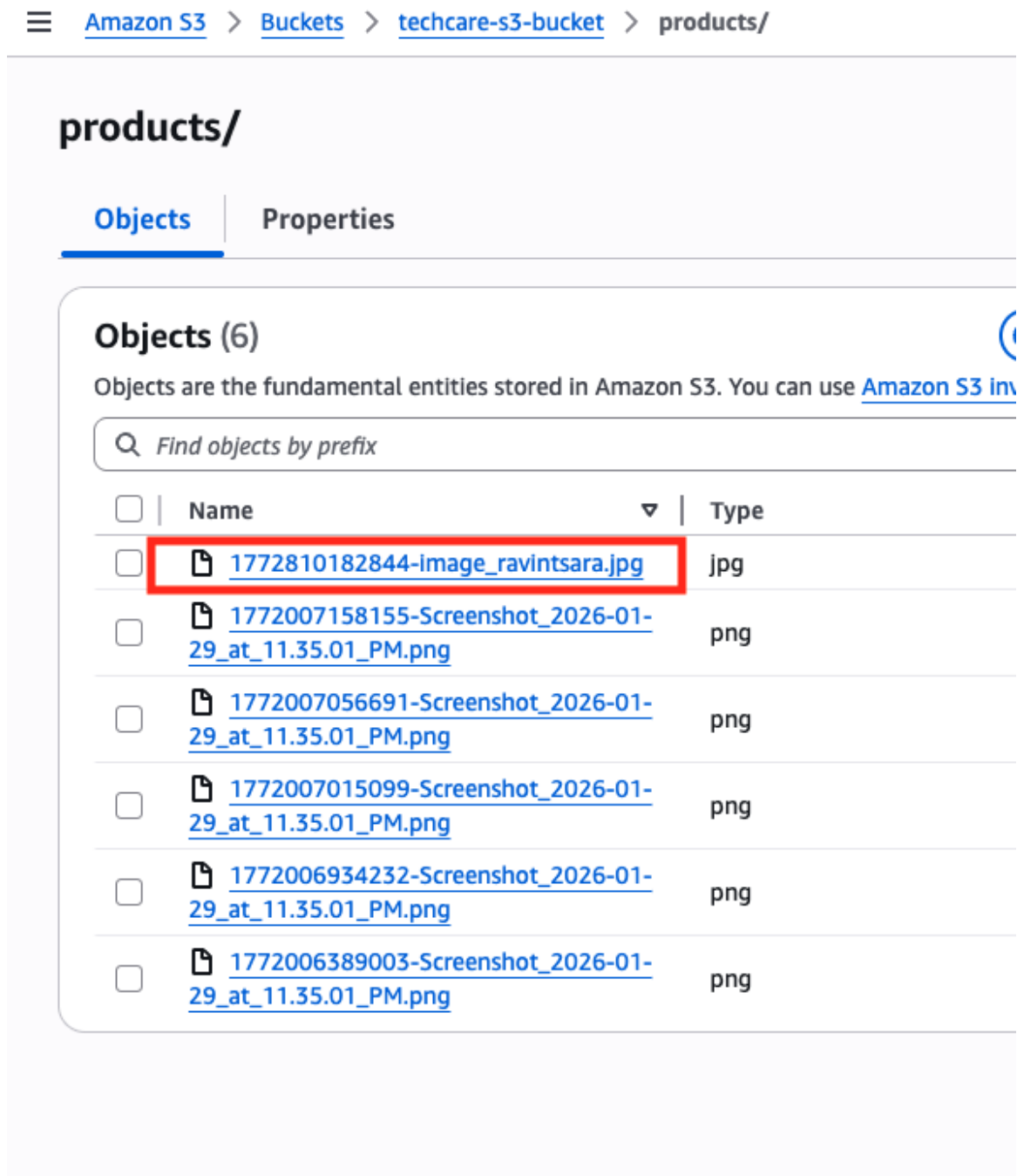


FIGURE 15 – Interface S3 : Validation de l’upload de l’image du produit

Cette externalisation du stockage physique garantit de meilleures performances d’affichage côté client en exploitant les capacités de diffusion d’un système de fichiers distribué.

IV.5 Perspectives d'évolutions et améliorations futures

Ce projet constitue une première étape solide qui ouvre la voie à de nombreuses optimisations.

IV.5.1 Orchestration avec Kubernetes

La prochaine étape logique serait la migration vers Kubernetes (K8s). Cela permettrait une auto-scalabilité dynamique des pods, une gestion plus fine des secrets et une meilleure résilience grâce au *Self-healing* (redémarrage automatique des conteneurs défectueux).

IV.5.2 Mise en place d'un Service Mesh

Pour résoudre la complexité du traçage et de la sécurité inter-services, l'adoption d'un Service Mesh (ex : Istio) permettrait d'automatiser le chiffrement TLS entre services (mtls) et de bénéficier de tableaux de bord de monitoring avancés sans modifier le code applicatif.

IV.5.3 CI/CD avancée

L'automatisation complète des tests et du déploiement via GitHub Actions ou GitLab CI renforcerait encore la rapidité de livraison de Techcare, en garantissant que chaque microservice est testé isolément avant sa mise en production.

IV.5.4 Optimisation des temps de réponse (Cache Redis)

Pour pallier le léger ralentissement observé lors des requêtes synchrones ("impôt de la distribution"), l'ajout d'une couche de cache en mémoire avec **Redis** au niveau de l'API Gateway serait une évolution majeure. Ainsi, lors des lectures fréquentes du catalogue, la Gateway pourrait renvoyer les données quasi instantanément (2 à 5 ms) sans solliciter le *Product Service* ni traverser le réseau.

IV.5.5 Transition vers gRPC pour les communications synchrones

Actuellement, les lectures inter-services transitent par des protocoles introduisant un surcoût de sérialisation textuelle (JSON). L'adoption de **gRPC** (Google Remote Procedure Call) pour ces échanges synchrones réduirait considérablement la taille des paquets grâce au format binaire Protobuf, et accélérerait la communication réseau.

Conclusion partielle

Ce chapitre a démontré que les objectifs initiaux de scalabilité, de sécurité et de disponibilité ont été atteints. Les tests fonctionnels via Postman confirment la fiabilité des mécanismes de protection et l'efficacité des échanges asynchrones via le broker de messages. Si l'architecture microservices apporte

une complexité technique supplémentaire, les bénéfices en termes d'agilité et de résilience justifient largement cet investissement pour l'évolution de la plateforme Techcare.

CONCLUSION GÉNÉRALE

Ce mémoire avait pour objectif de répondre à une problématique centrale : comment faire évoluer une plateforme pharmaceutique comme Techcare, confrontée aux limites structurelles d'une architecture monolithique, afin de garantir la scalabilité, la résilience et la maintenabilité requises par sa croissance ?

Pour y répondre, nous avons adopté une démarche scientifique et progressive. Le premier chapitre a posé le contexte du projet et les exigences fonctionnelles et non-fonctionnelles. Le deuxième chapitre a fourni les fondements théoriques indispensables à toute architecture distribuée — le théorème CAP, les patterns de résilience, le Domain-Driven Design — et a abouti à une conception rigoureuse de l'architecture cible. Le troisième chapitre a concrétisé cette conception en une implémentation technique avec NestJS, RabbitMQ, Docker et les services cloud AWS. Enfin, le quatrième chapitre a offert un regard critique sur les résultats obtenus et les perspectives d'évolution.

Les objectifs fixés en début de projet ont été atteints :

- **Scalabilité** : L'architecture microservices permet désormais de faire évoluer indépendamment chaque domaine métier. Le *Product Service* peut absorber des pics de consultation sans impacter le traitement des paiements.
- **Résilience** : Le pattern Saga avec compensation garantit l'intégrité des données distribuées. La panne d'un service n'entraîne plus l'indisponibilité totale de la plateforme.
- **Maintenabilité** : L'architecture hexagonale adoptée au sein de chaque service et le langage ubiquiste du DDD ont rendu le code aligné sur le métier pharmaceutique, réduisant considérablement les risques de régression lors des évolutions.
- **Sécurité** : La stratégie Zero Trust, la propagation d'identité via JWT et le contrôle d'accès par rôles (RBAC) offrent un niveau de sécurité adapté à un contexte médical sensible.

Néanmoins, cette transition n'est pas sans limites. La complexité opérationnelle accrue, la gestion de la consistance éventuelle, ainsi que le léger compromis sur la latence réseau (le fameux "impôt de la distribution") représentent des défis concrets inhérents aux systèmes distribués. Ces limites, identifiées et mesurées lors de nos phases de test, constituent une véritable feuille de route pour les évolutions futures. Les perspectives d'optimisation sont claires : l'orchestration dynamique avec Kubernetes, l'adoption d'un *Service Mesh* pour déléguer la sécurité inter-services et rendre le réseau observable sans modifier le code applicatif, ou encore l'intégration d'un cache Redis et du protocole gRPC pour pulvériser les temps de réponse.

En définitive, ce projet démontre qu'une migration vers les microservices ne doit pas être menée comme une fin en soi, mais comme une réponse mesurée à des besoins métiers concrets et quantifiables. Techcare dispose aujourd'hui d'un socle technologique moderne, évolutif et aligné sur les meilleures

pratiques de l'industrie logicielle, prêt à accompagner la croissance de la plateforme pharmaceutique de demain.

BIBLIOGRAPHIE

- [1] **NEWMAN, Sam.** *Building Microservices : Designing Fine-Grained Systems*. O'Reilly Media, 2015.
- [2] **EVANS, Eric.** *Domain-Driven Design : Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003.
- [3] **RICHARDSON, Chris.** *Microservices Patterns : With examples in Java*. Manning Publications, 2018.
- [4] **KLEPPMANN, Martin.** *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

WÉBOGRAPHIE

- [W1] **NestJS Framework officiel.** *Documentation officielle de NestJS.* Disponible sur : <https://nestjs.com/> (Consulté en Janvier/Février 2026).
- [W2] **RabbitMQ Documentation.** *Messaging that just works.* Disponible sur : <https://www.rabbitmq.com/documentation.html> (Consulté au long du projet).
- [W3] **Docker Documentation.** *Docker Docs.* Disponible sur : <https://docs.docker.com/> (Le conteneur pour le développement moderne).
- [W4] **Microservices.io.** *Microservice Architecture.* Disponible sur : <https://microservices.io/> (Portail de Chris Richardson sur les patterns microservices).
- [W5] **Drizzle ORM.** *TypeScript ORM that is both fast and type safe.* Disponible sur : <https://orm.drizzle.team/> (Consulté pour la persistance PostgreSQL).

ANNEXES

Annexe A — Documentation Complète de l'API REST

A.1 Informations générales

- **Base URL** : `http://localhost:4000` (développement)
- **Format** : `application/json` pour toutes les réponses
- **Authentification** : JWT Bearer Token dans le header `Authorization`
- La colonne **Auth** indique si la route est protégée (Oui) ou publique (Non).

A.2 Module Authentification (`/auth`)

Méthode	Endpoint	Description	Auth
POST	<code>/auth/signup</code>	Créer un compte utilisateur	Non
POST	<code>/auth/signin</code>	Se connecter, retourne un JWT	Non
GET	<code>/auth/me</code>	Retourne le profil de l'utilisateur connecté	Oui

TABLE 7 – Endpoints du module Authentification

Corps de POST `/auth/signup` et POST `/auth/signin` :

```
{ "email": "user@techcare.mg", "password": "motdepasse" }
```

Réponse de POST `/auth/signin` :

```
{ "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

A.3 Module Produits (`/products`)

Méthode	Endpoint	Description	Auth
GET	<code>/products</code>	Liste complète du catalogue	Non
GET	<code>/products/:id</code>	Détail d'un produit par ID	Non
POST	<code>/products</code>	Créer un produit (multipart/form-data avec image)	Oui
PUT	<code>/products/:id</code>	Mettre à jour un produit	Oui
DELETE	<code>/products/:id</code>	Supprimer un produit	Oui

TABLE 8 – Endpoints du module Produits

Champs pour POST/PUT `/products` (multipart/form-data) :

name : string - Nom du produit
 description : string - Description
 price : number - Prix unitaire
 category : string - ObjectId de la catégorie
 image : File - Fichier image (jpg, png)

A.4 Module Catégories (/category)

Méthode	Endpoint	Description	Auth
GET	/category	Toutes les catégories	Non
GET	/category/:id	Une catégorie par ID	Non
POST	/category	Créer une catégorie	Oui
PUT	/category/:id	Modifier	Oui
DELETE	/category/:id	Supprimer	Oui

TABLE 9 – Endpoints du module Catégories

A.5 Module Inventaire (/inventory)

Méthode	Endpoint	Description	Auth
GET	/inventory	Stock de tous les produits	Non
GET	/inventory/:productId	Stock d'un produit spécifique	Non
POST	/inventory/update	Mettre à jour le stock	Oui

TABLE 10 – Endpoints du module Inventaire

Corps de POST /inventory/update :

```
{ "productId": "6502d...", "quantity": 50 }
```

A.6 Module Panier (/cart)

Méthode	Endpoint	Description	Auth
POST	/cart/add	Ajouter un article au panier	Non
GET	/cart/:userId	Consulter le panier	Non
PUT	/cart/update	Modifier la quantité	Non
DELETE	/cart/remove	Retirer un article	Non
DELETE	/cart/clear/:userId	Vider le panier	Non

TABLE 11 – Endpoints du module Panier

Corps de POST /cart/add :

```
{
  "userId": "user123", "productId": "6502d...",
  "quantity": 2,      "price": 3.50,
  "name": "Doliprane", "image": "https://..."
}
```

A.7 Module Commandes (/order)

Méthode	Endpoint	Description	Auth
POST	/order/create	Créer une commande (déclenche Saga)	Non
GET	/order	Toutes les commandes (admin)	Non
GET	/order/:orderId	Commande par ID	Non
GET	/order/user/:userId	Commandes d'un utilisateur	Non
PUT	/order/status	Mettre à jour le statut	Non
PUT	/order/cancel/:orderId	Annuler (déclenche compensation)	Non

TABLE 12 – Endpoints du module Commandes

Corps de PUT /order/status :

```
{
  "orderId": "6981b...", "status": "shipped",
  "trackingNumber": "TRACK123456"
}
```

Valeurs de status : PENDING | CONFIRMED | shipped | DELIVERED | CANCELLED

A.8 Module Paiement (/payment)

Méthode	Endpoint	Description	Auth
POST	/payment/create-intent	Créer un PaymentIntent Stripe	Non
POST	/payment/confirm	Confirmer le paiement	Non
POST	/payment/refund	Rembourser	Non
GET	/payment/intent/:id	Détails d'un PaymentIntent	Non

TABLE 13 – Endpoints du module Paiement

Corps de POST /payment/create-intent :

```
{ "orderId": "6981b...", "amount": 59.98, "currency": "eur" }
```

Réponse :

```
{ "clientSecret": "pi_30..._secret...", "paymentIntentId": "pi_30..." }
```

A.9 Module Fichiers (/files)

Méthode	Endpoint	Description	Auth
POST	/files/upload	Upload un fichier vers AWS S3	Oui

TABLE 14 – Endpoints du module Fichiers

Form-data : champ file (binaire) + champ optionnel folder (ex : products).

Réponse :

```
{ "url": "https://bucket.s3.amazonaws.com/products/1708456723-image.jpg" }
```

A.10 Codes d’erreur HTTP

Code	Signification
200	Succès
201	Ressource créée avec succès
400	Données invalides / champ manquant dans le body
401	Token JWT absent, invalide ou expiré
403	Rôle insuffisant pour accéder à cette ressource
404	Ressource introuvable (produit, commande, etc.)
500	Erreur interne du serveur
503	Microservice ou broker RabbitMQ temporairement indisponible

TABLE 15 – Codes d’erreur HTTP retournés par l’API Gateway

Table des matières

REMERCIEMENTS	i
RÉSUMÉ	ii
ABSTRACT	iii
LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES	vi
GLOSSAIRE	viii
LISTE DES TABLEAUX	ix
LISTE DES FIGURES	x
INTRODUCTION	1
CHAPITRE I - CONTEXTE GÉNÉRAL DU PROJET	2
I.1. Présentation de l'INSI	2
I.1.1. Information d'ordre général	2
I.1.2. Missions et historiques	2
I.1.3. Organigramme de l'INSI	3
I.1.4. Domaine de spécialisation	4
I.1.5 Architectures des formations pédagogiques	5
I.1.6. Relations de l'INSI avec les entreprises et les organismes	6
I.1.7. Partenariat au niveau international	6
I.1.8. Parcours professionnels des diplômés	7
I.1.9. Ressources humaines	8
I.2. Contexte professionnel et entreprise d'accueil	9
I.2.1. Présentation de l'entreprise d'accueil	9
I.2.2. Contexte du projet	9
I.3 Problématique et justification du projet	10
I.4 Objectifs et périmètre du projet	10
I.5 Méthodologie et plan de travail	10
I.5.1 Méthodologie de gestion de projet : Scrum	11
I.5.2 Plan de travail et étapes de réalisation	11
I.5.3 Outils de collaboration et environnement technique	11
CHAPITRE II - ANALYSE ET CONCEPTION	13
II.1 Analyse de l'existant et État de l'Art	13

II.1.1 Évolution des architectures logicielles : Un parcours historique	14
II.1.2 L'Architecture Monolithique : Analyse du Modèle Traditionnel	14
II.1.2.1 Caractéristiques et Modèle de Déploiement	14
II.1.2.2 Atouts Techniques et Opérationnels	15
II.1.2.3 Limites Structurelles et le phénomène "Big Ball of Mud"	15
II.2 Fondements Théoriques et Contraintes Distribuées	16
II.2.1 Le Théorème CAP : Le dilemme de la distribution	16
II.2.2 Patterns de Résilience : Prévenir l'effet domino	16
II.3 Découpage Métier : Le Domain-Driven Design (DDD)	16
II.4 Modélisation des Microservices et Communications	17
II.4.1 API Gateway et Backend-for-Frontend (BFF)	17
II.4.2 Chorégraphie : La force de l'asynchronisme	17
II.5 Architecture de Persistance et Base de Données	18
II.5.1 Justification des choix technologiques	19
II.5.1.1 Justification du choix de PostgreSQL	19
II.5.1.2 Justification du choix de MongoDB	19
II.5.1.3 Justification des technologies applicatives et services tiers	20
II.5.2 Gestion des transactions distribuées : Pattern Saga	20
Définition formelle	20
Les deux approches de Saga	21
Exemple concret : Commande de médicament	21
II.6 Modélisation de la Sécurité : Zero Trust Architecture	22
II.6.1 Architecture de sécurité en couches	22
II.6.2 Gestion des rôles et permissions	23
II.7 Modélisation des Flux et Cas d'Usage	23
II.7.1 Processus métier de commande	23
CHAPITRE III - RÉALISATION	27
III.1 Organisation du Projet et Arborescence	27
III.2 Environnement Technologique et Outils	27
III.1.1 Framework NestJS	27
III.1.2 Communication : RabbitMQ	28
III.1.3 Bases de données et ORM	28
III.2.1 Gestion de la Configuration et des Secrets	28
III.3 Architecture Interne et Patrons de Conception	29
III.2.1 Organisation en couches	30
III.2.2 Flux de données et DTOs	30
III.3.3 Tableau Récapitulatif des Microservices	30
III.4 Développement de l'API Gateway	31
III.4.1 Architecture et Responsabilités	31

III.4.2	Coordination Inter-services	31
III.4.3	Gestion des Exceptions (RpcExceptionFilter)	32
III.4.4	Contrat d'API : Référence des Endpoints	33
III.5	Implémentation des Microservices Core	34
III.5.1	Product Manager : Catalogue et Catégories	34
III.5.2	Inventory Manager : Transactions PostgreSQL	35
III.5.3	File Manager : Stockage S3	35
III.5.4	Order Manager et la Saga de Compensation	36
III.5.5	Payment Manager et Notification Manager	38
III.6	Sécurisation et Gestion des Accès	38
III.6.1	Stratégie d'Authentification JWT	38
III.6.2	Contrôle d'accès par rôles (RBAC)	39
III.6.3	Propagation de l'identité	39
III.7	Orchestration et DevOps	40
III.6.1	Construction des images : Multi-stage Building	40
III.6.2	Orchestration : Docker Compose	41
III.6.3	Workflow de développement	41
III.8	Stratégie de Test et Validation	42
CHAPITRE IV	- RÉSULTATS ET ANALYSE	43
IV.1	Évaluation des performances et de la scalabilité	43
IV.1.1	Temps de réponse et latence	43
IV.1.2	Montée en charge (Scalabilité)	43
IV.2	Analyse de la qualité logicielle et maintenabilité	43
IV.2.1	Indépendance des déploiements	43
IV.2.2	Clarté du code et isolation des fautes	44
IV.2.3	Mutualisation du code et sécurité du typage	44
IV.3	Bilan critique et limites de la solution	44
IV.3.1	Complexité opérationnelle	44
IV.3.2	Consistance éventuelle	44
IV.3.3	Robustesse du Broker de messages (SPOF)	44
IV.4	Validation fonctionnelle via Postman	45
IV.4.1	Test de l'authentification (Sign-in)	45
IV.4.2	Validation de la protection des routes (/auth/me)	47
IV.4.3	Consultation du catalogue (Performance et "Impôt de la distribution")	47
IV.4.4	Communication asynchrone et événements (RabbitMQ)	49
IV.4.5	Validation des données et robustesse (Shared Library)	50
IV.4.6	Gestion des fichiers et stockage S3 (File Service)	51
IV.5	Perspectives d'évolutions et améliorations futures	53
IV.5.1	Orchestration avec Kubernetes	53

IV.5.2 Mise en place d'un Service Mesh	53
IV.5.3 CI/CD avancée	53
IV.5.4 Optimisation des temps de réponse (Cache Redis)	53
IV.5.5 Transition vers gRPC pour les communications synchrones	53
CONCLUSION GÉNÉRALE	55
BIBLIOGRAPHIE	xi
WÉBOGRAPHIE	xii
ANNEXES	xiii
Annexe A — Documentation de l'API REST	xiii
TABLE DES MATIÈRES	xvii